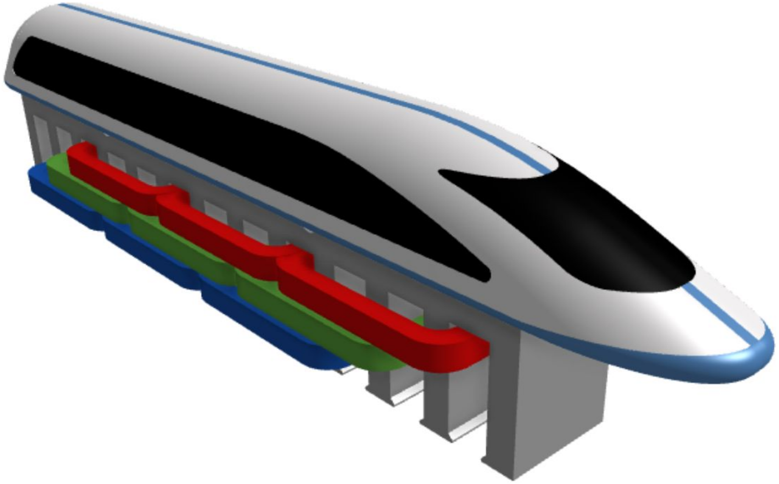
Mini Maglev

Precise Speed Control of a Linear Induction Motor



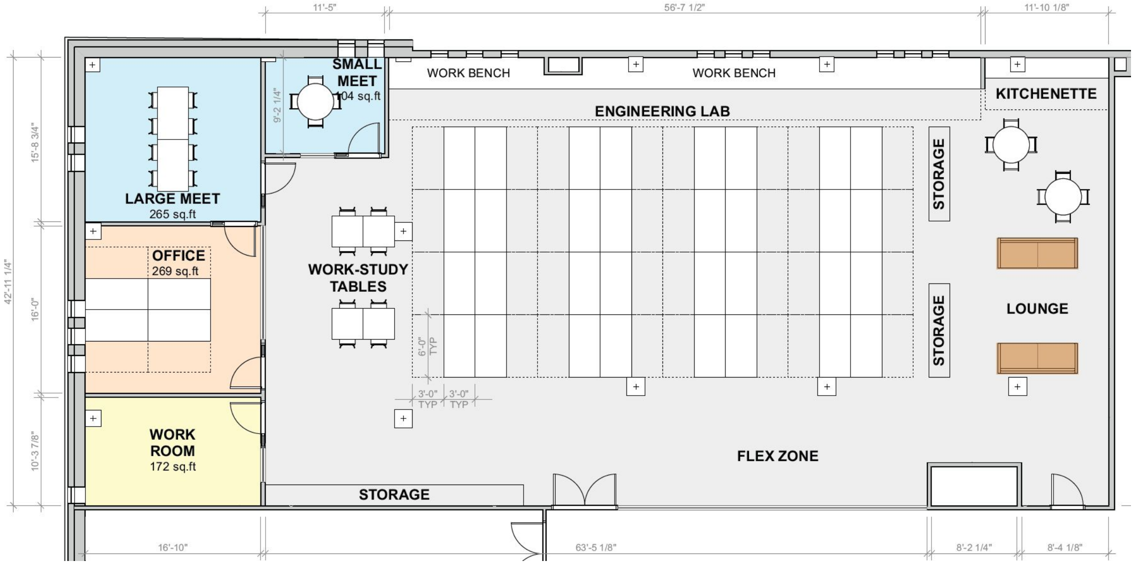
   

Team 2504: Ada Li, Amy Li, Cyrus Young, Dhruv Arun

Sponsor: ENPH Project Lab, UBC

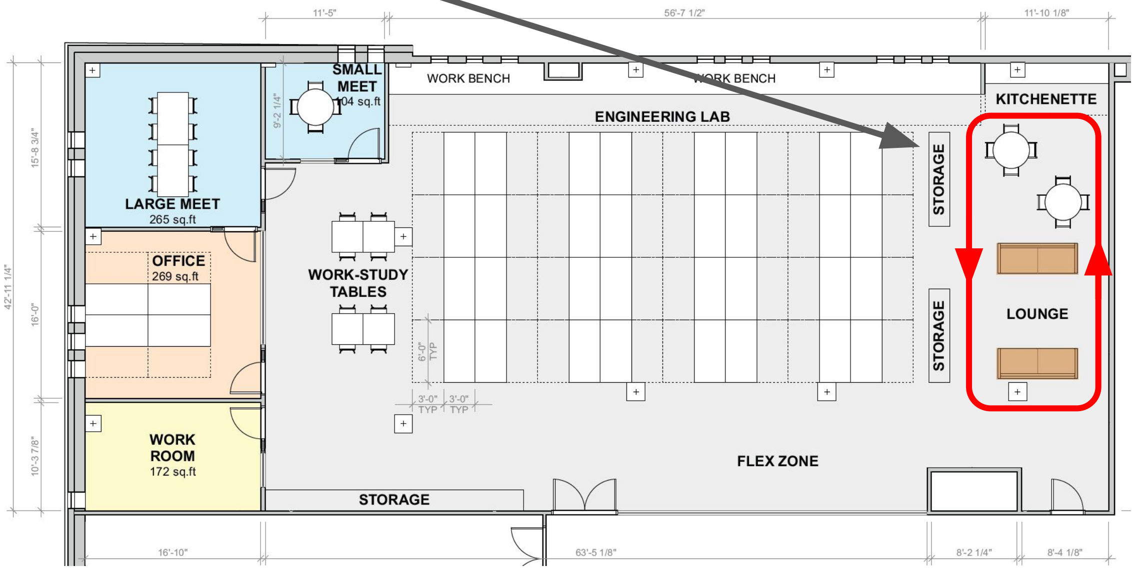
Motivation

The project lab expansion is missing one thing…



Motivation

… a mini maglev track.

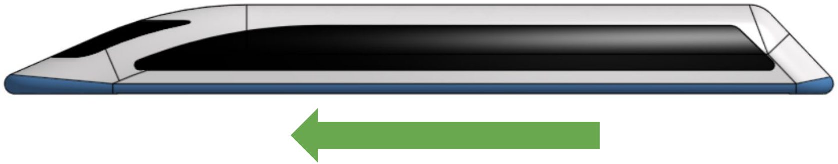


Scope of project

- **Maglev**: train levitated above rails by magnetic repulsion
- **Induction motor**: currents in the rotor are induced by time-varying magnetic fields from the stator
- **Linear**: produces linear motion

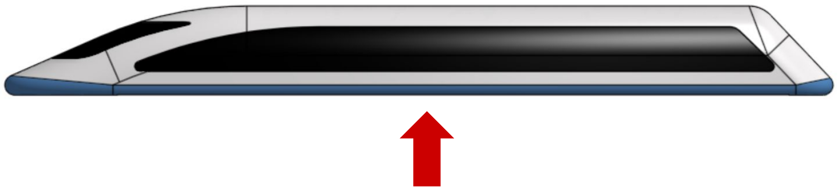
This year:

- Build a linear induction motor and precisely control its thrust output



Next year:

- Stable levitation



Simplified System Level Diagram

Speed Input

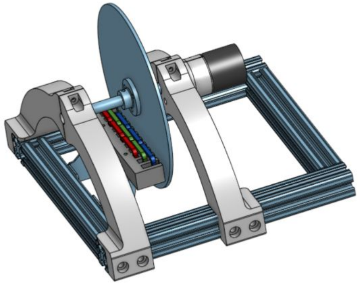
High Voltage Power Supply

Simplified System Level Diagram

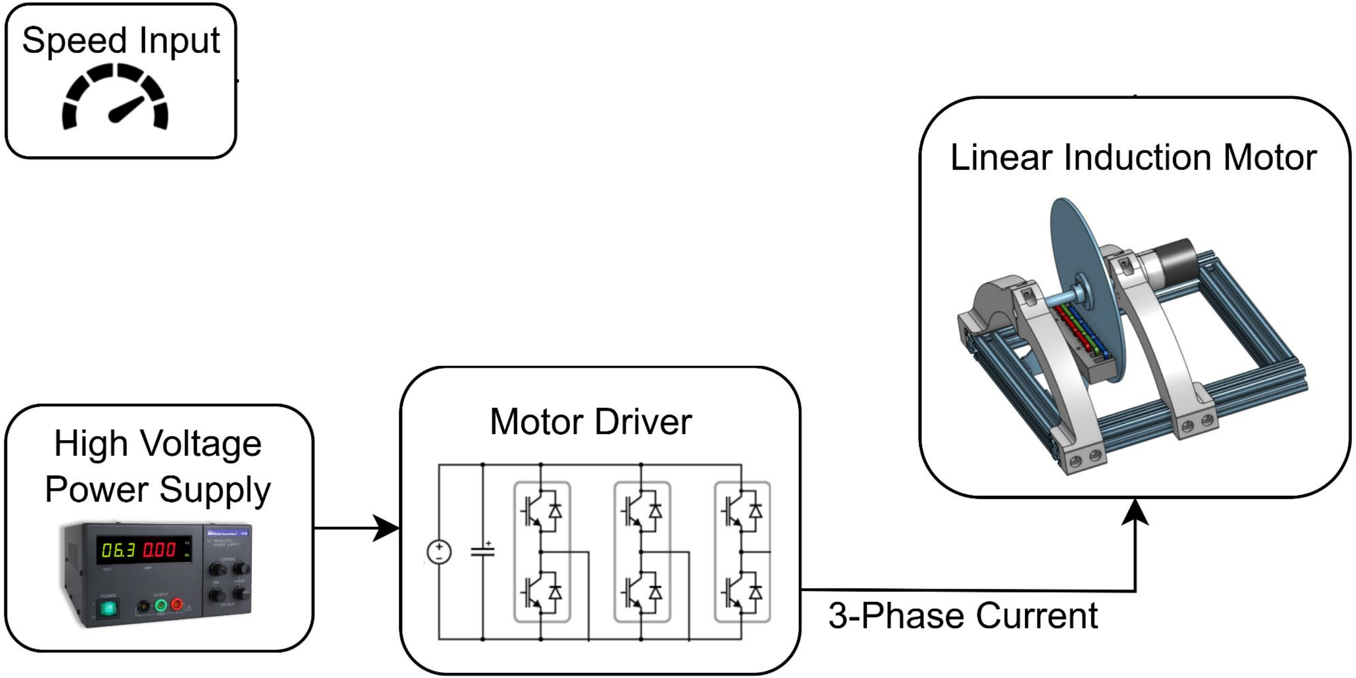
Speed Input

High Voltage
Power Supply

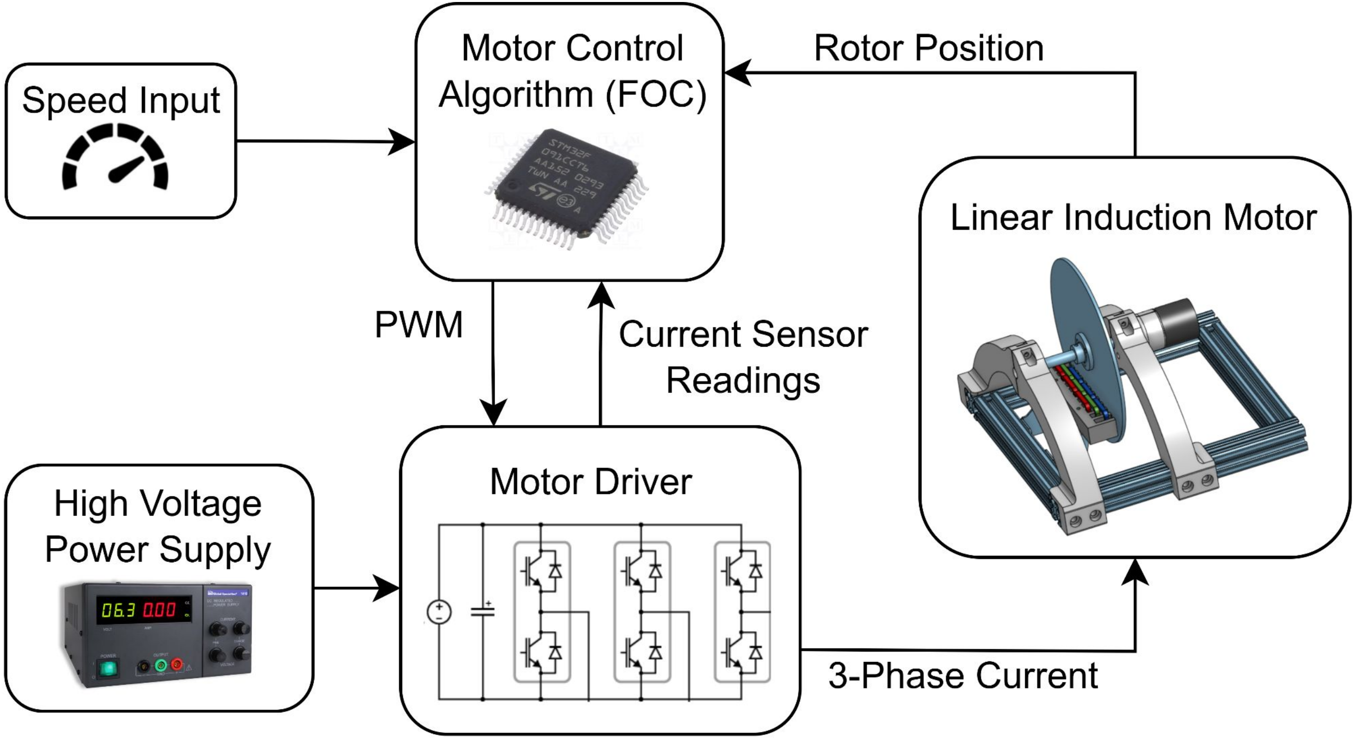
Linear Induction Motor



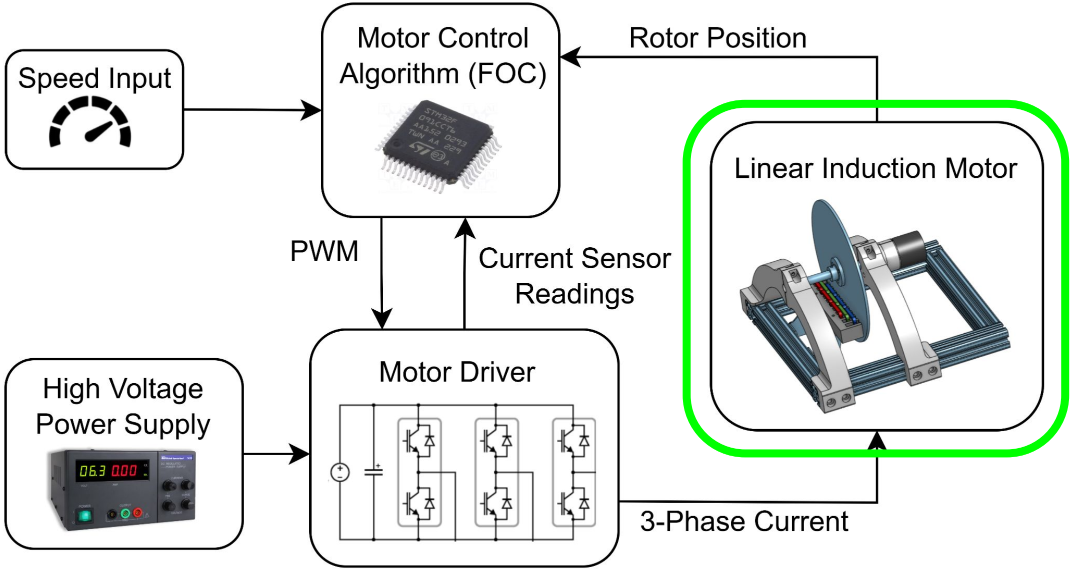
Simplified System Level Diagram



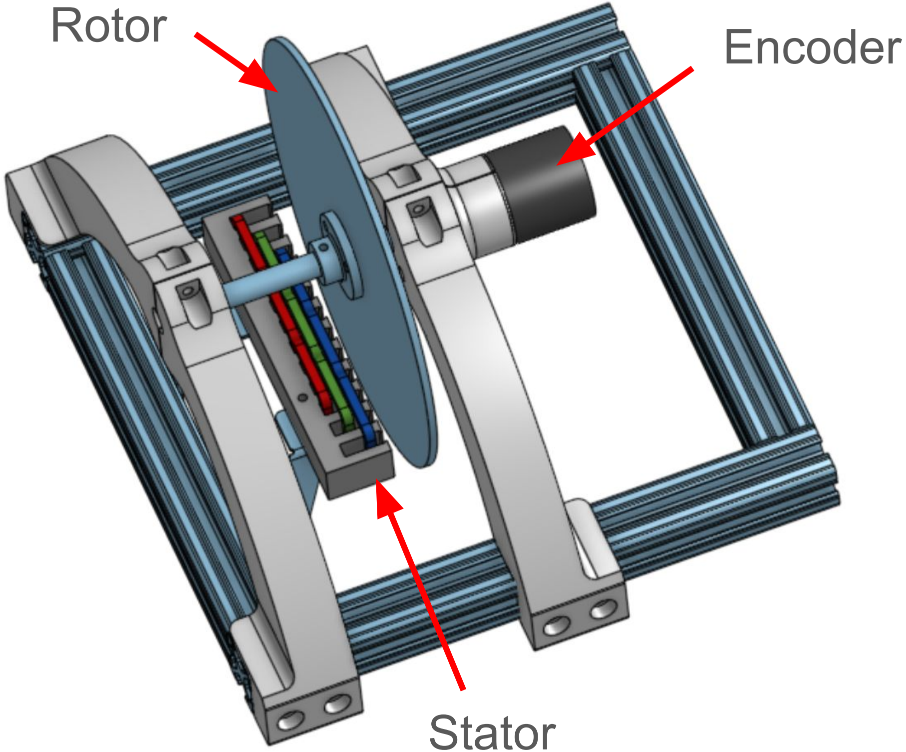
Simplified System Level Diagram

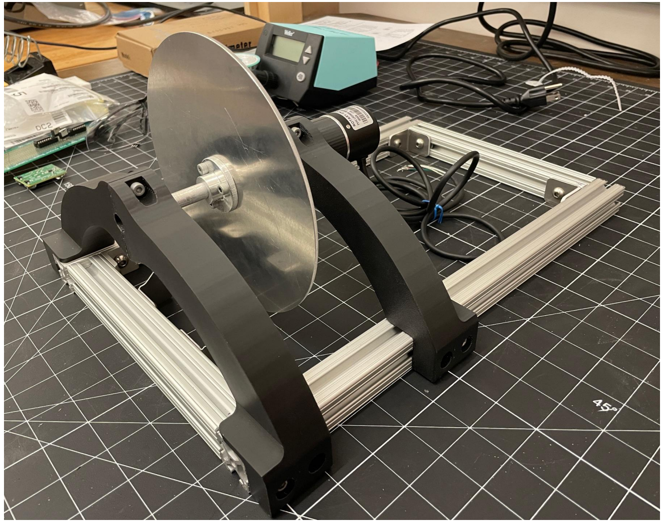


Rotor/Stator Design and Thrust Generation

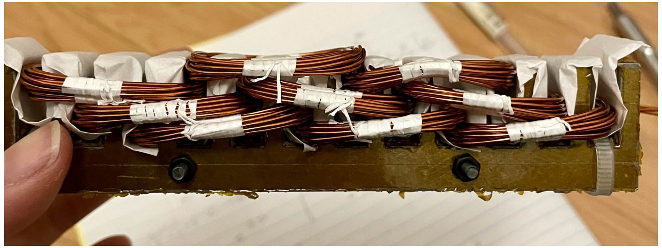


Linear Induction Motor Test Stand



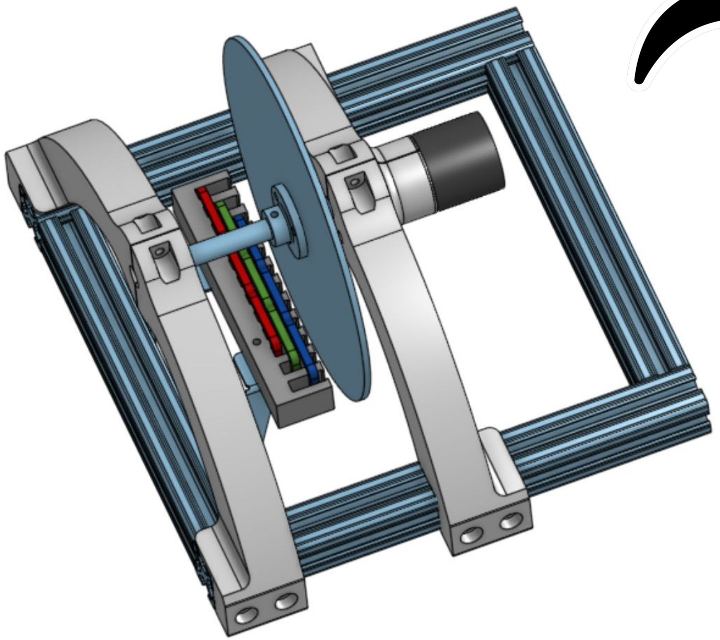


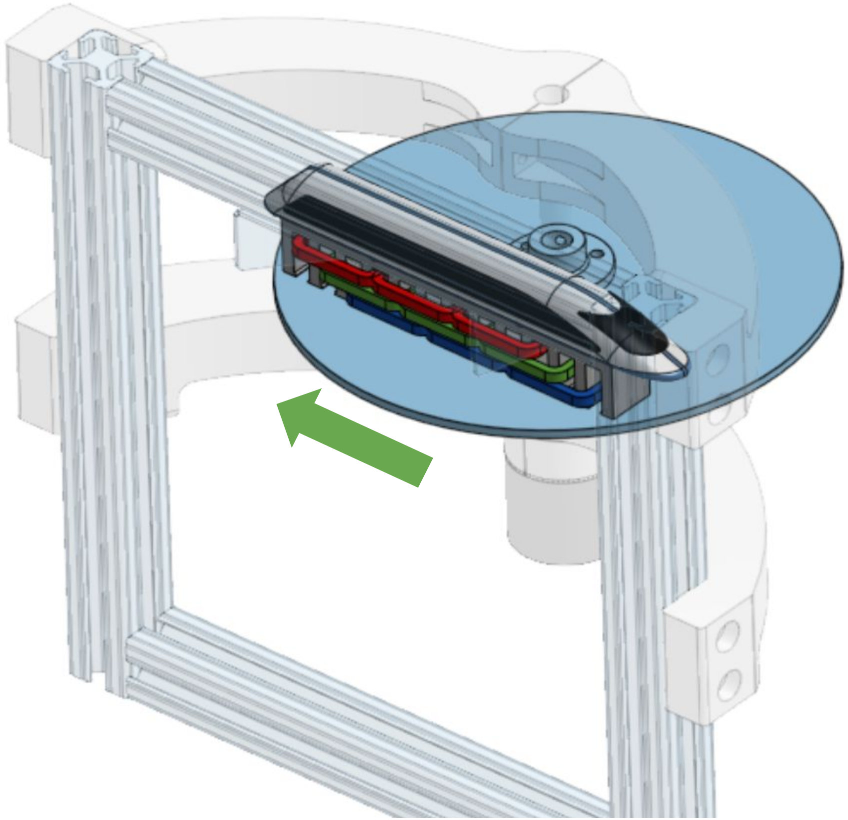
Test stand with rotor



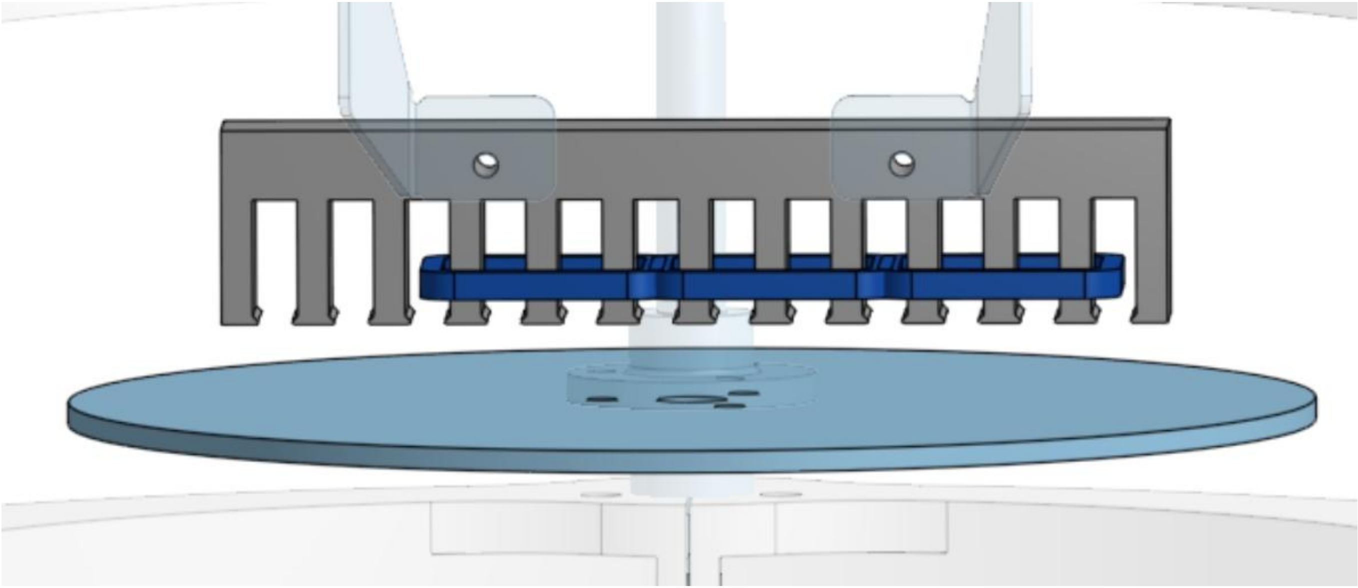
Stator

Linear Induction Motor Test Stand

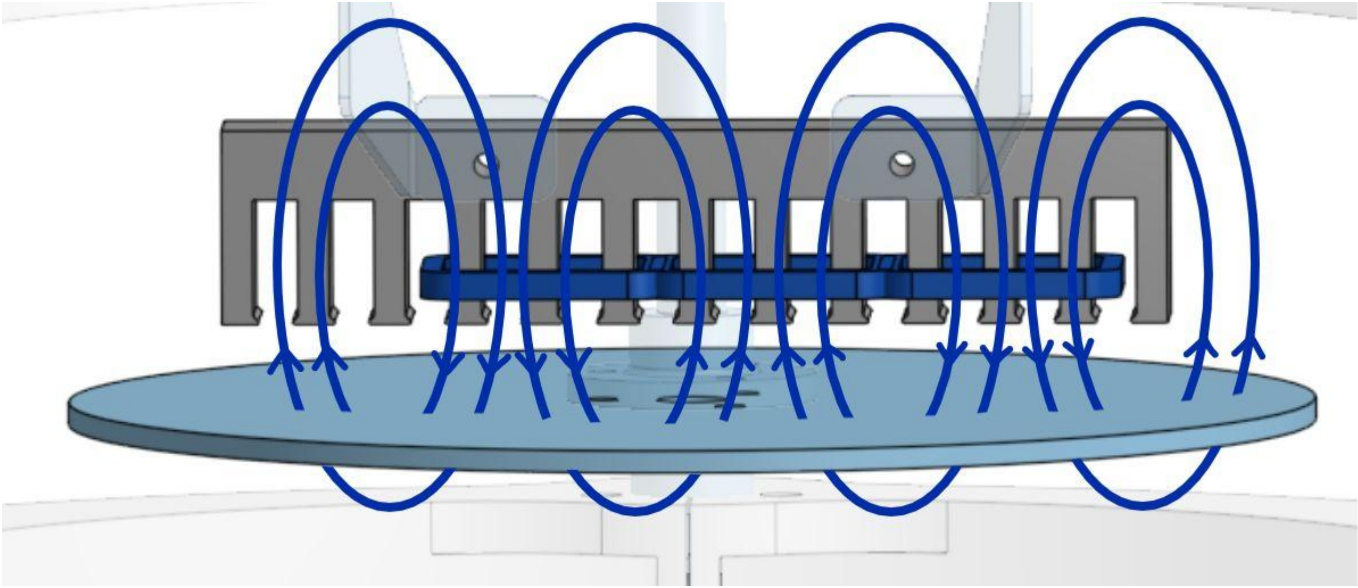




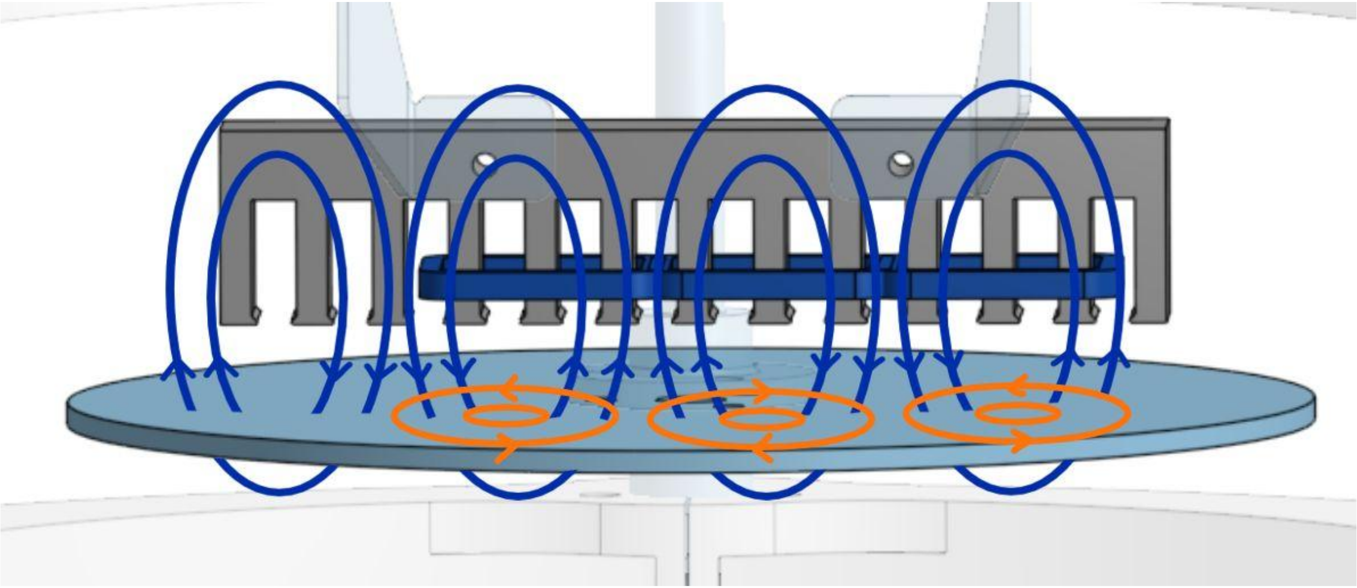
Producing Thrust



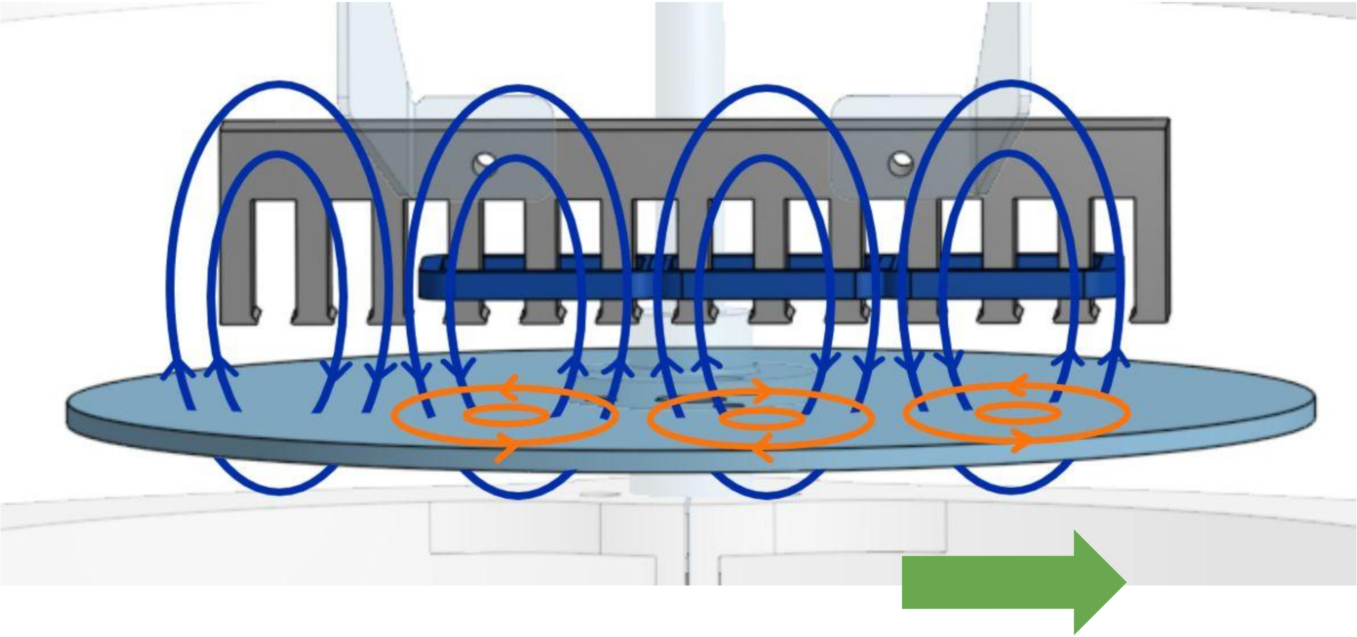
Producing Thrust



Producing Thrust

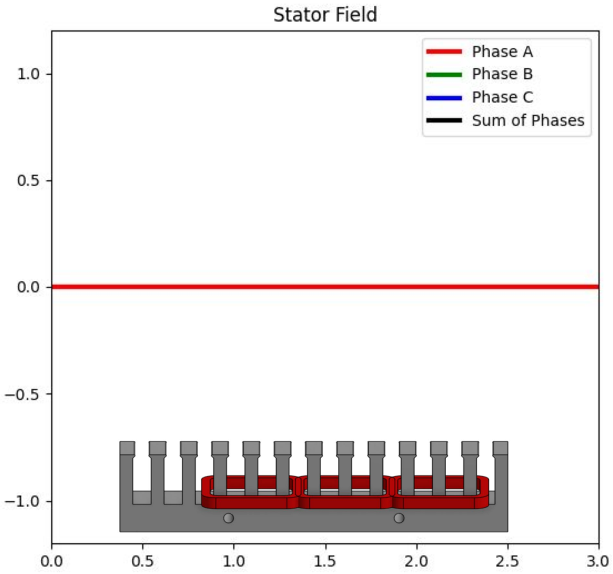


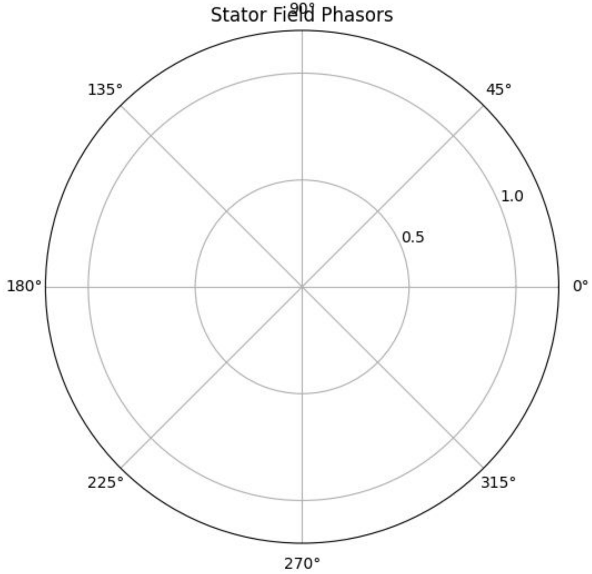
Producing Thrust



The Travelling Flux Wave

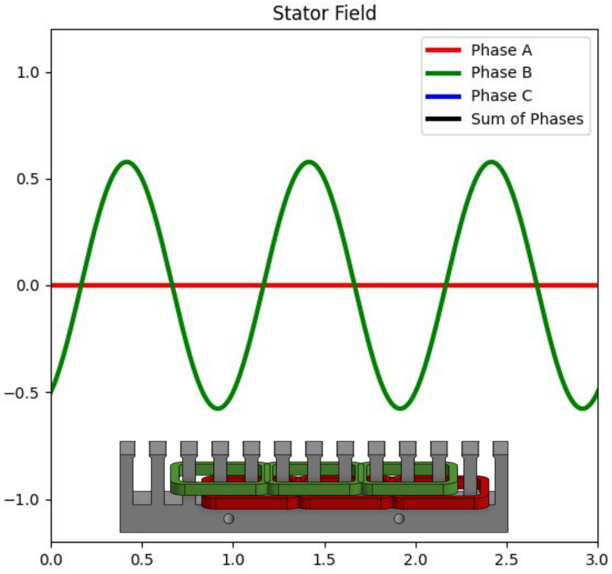
- Sinusoidally varying currents in 3 stator phases displace in space produce a travelling flux wave

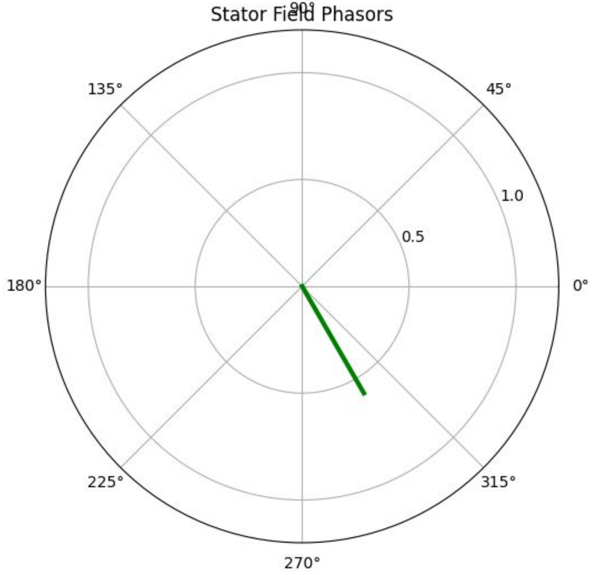




The Travelling Flux Wave

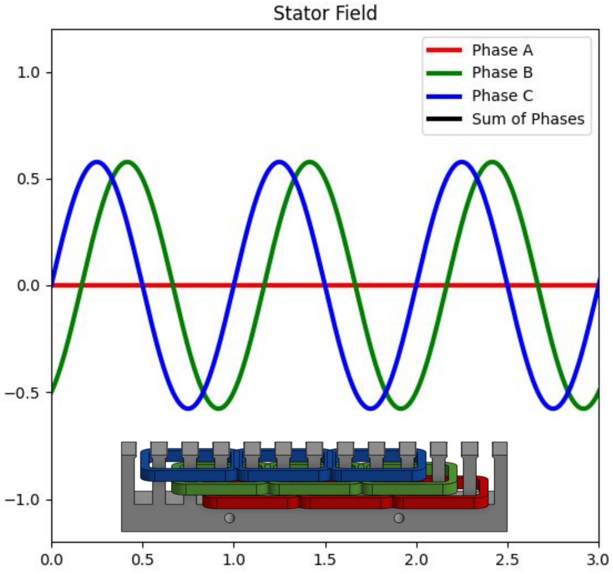
- Sinusoidally varying currents in 3 stator phases displace in space produce a travelling flux wave

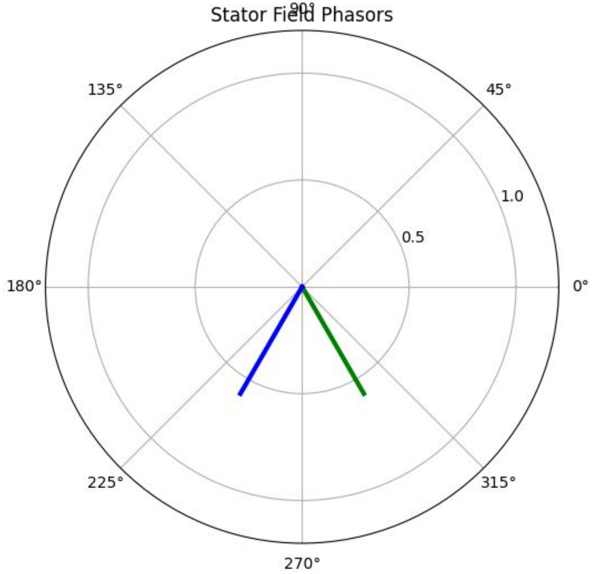




The Travelling Flux Wave

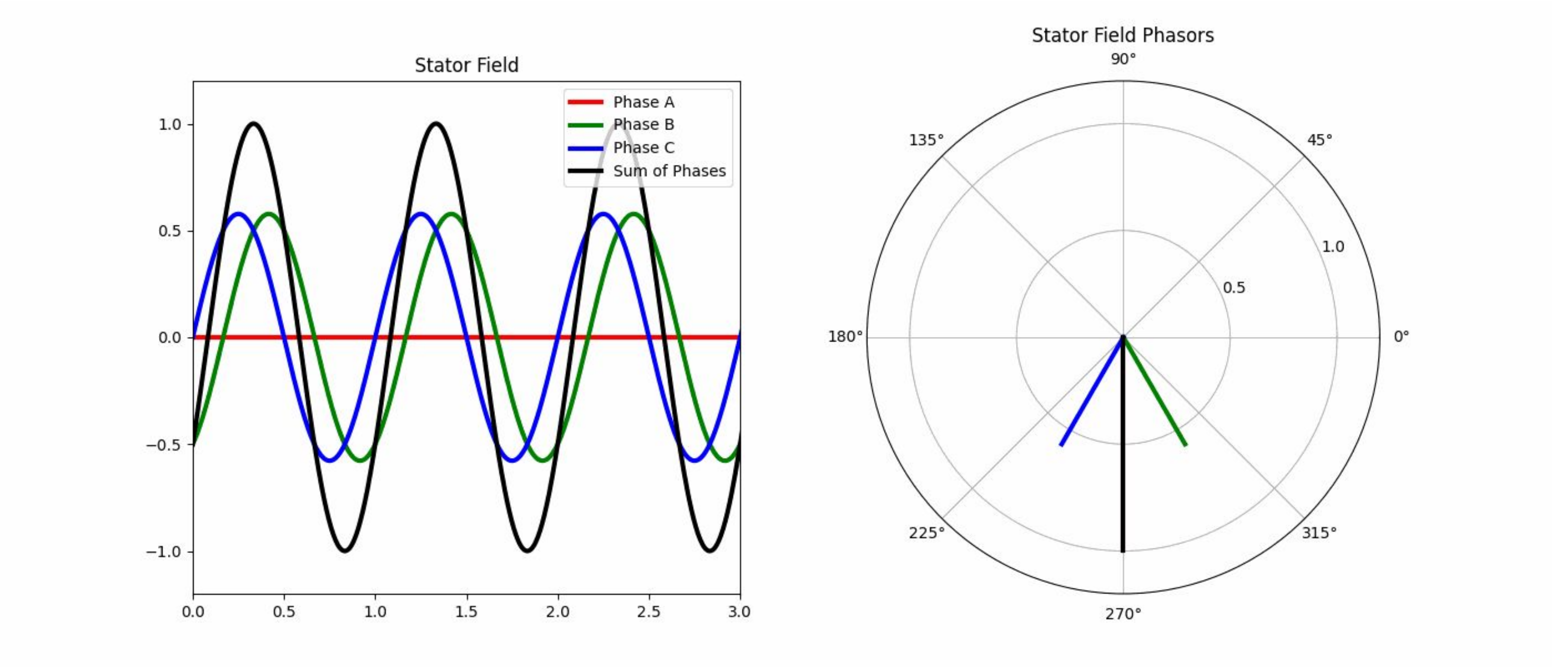
- Sinusoidally varying currents in 3 stator phases displace in space produce a travelling flux wave



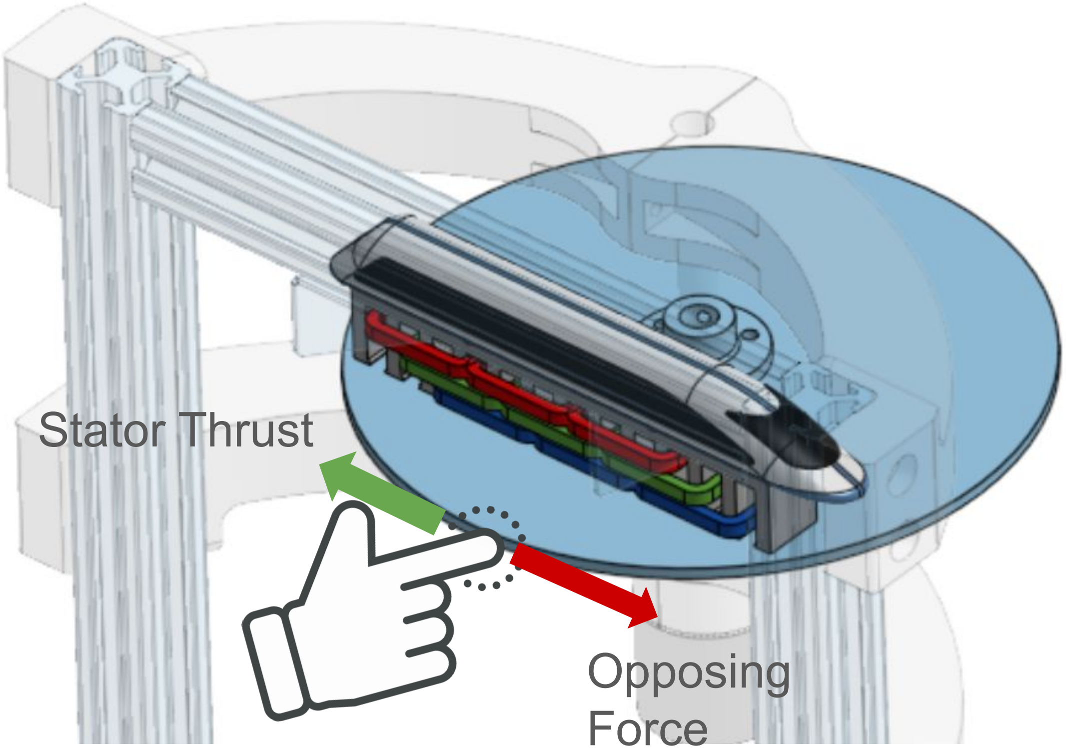


The Travelling Flux Wave

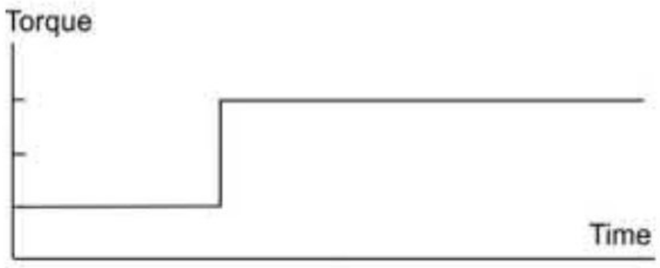
- Sinusoidally varying currents in 3 stator phases displace in space produce a travelling flux wave



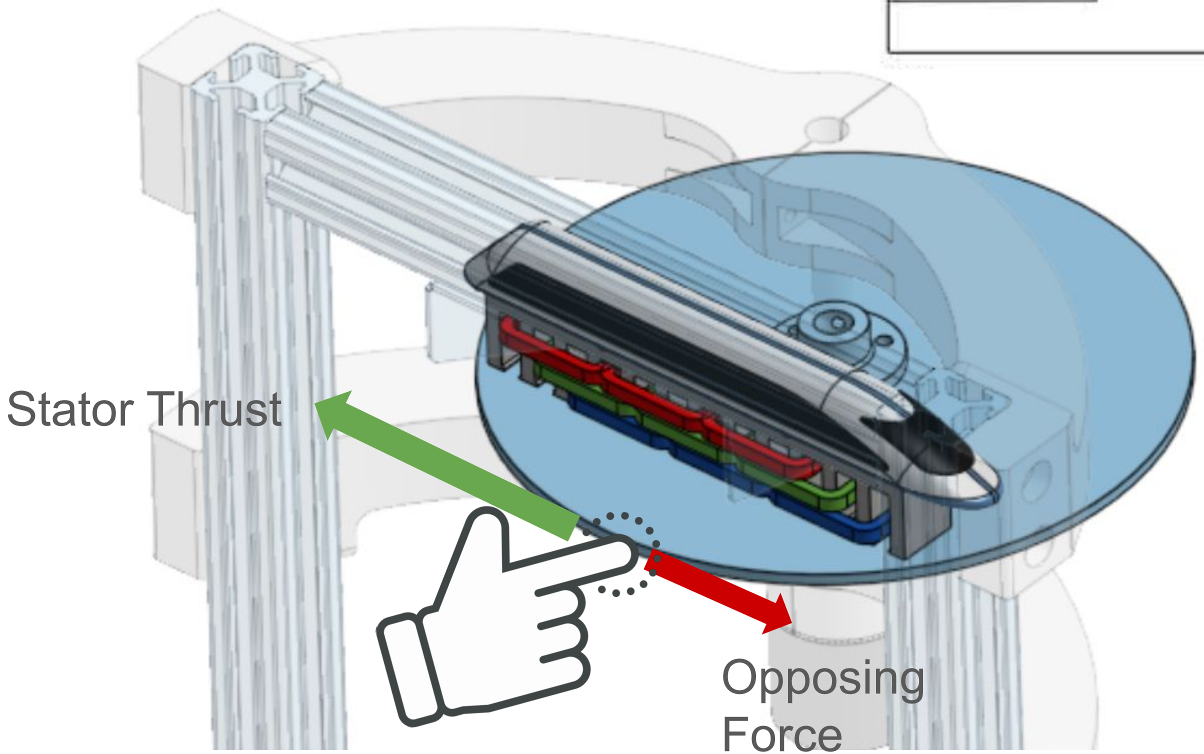
Instantaneous Thrust Control



Instantaneous Thrust Control

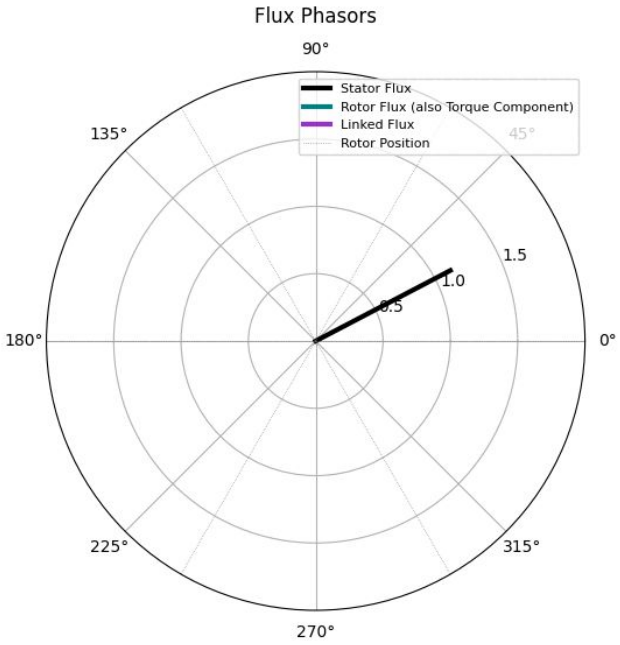


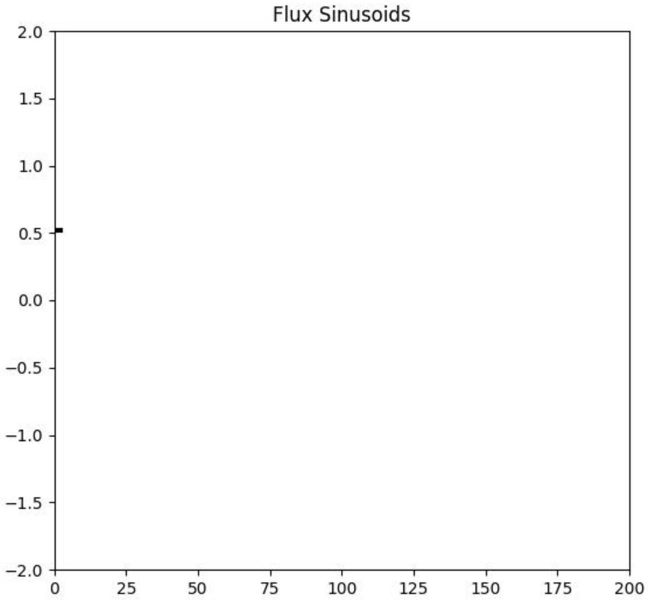
$$T = \psi_R I_R$$



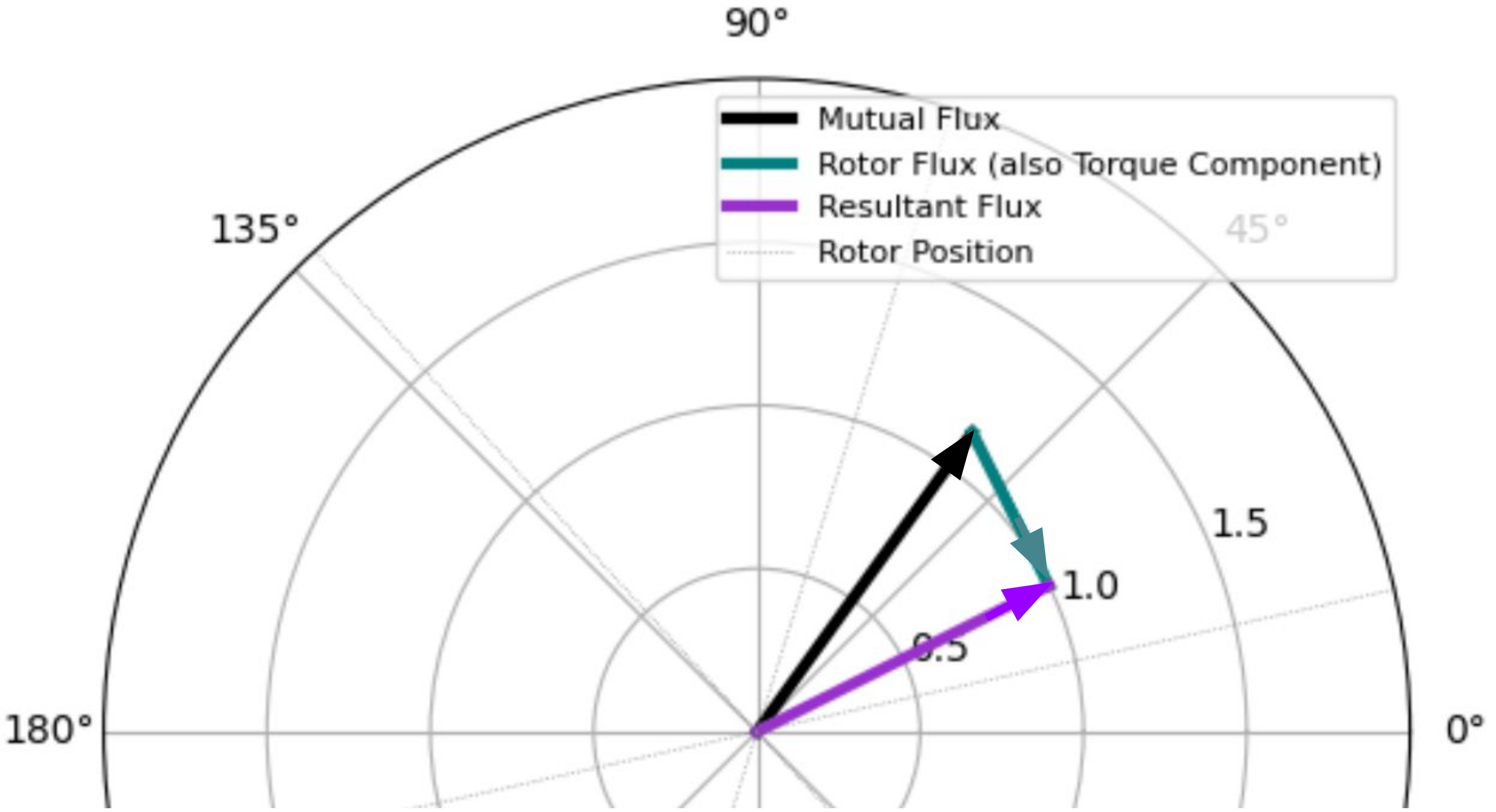
Instantaneous Thrust Control (FOC)

Recall the travelling flux wave of the stator:

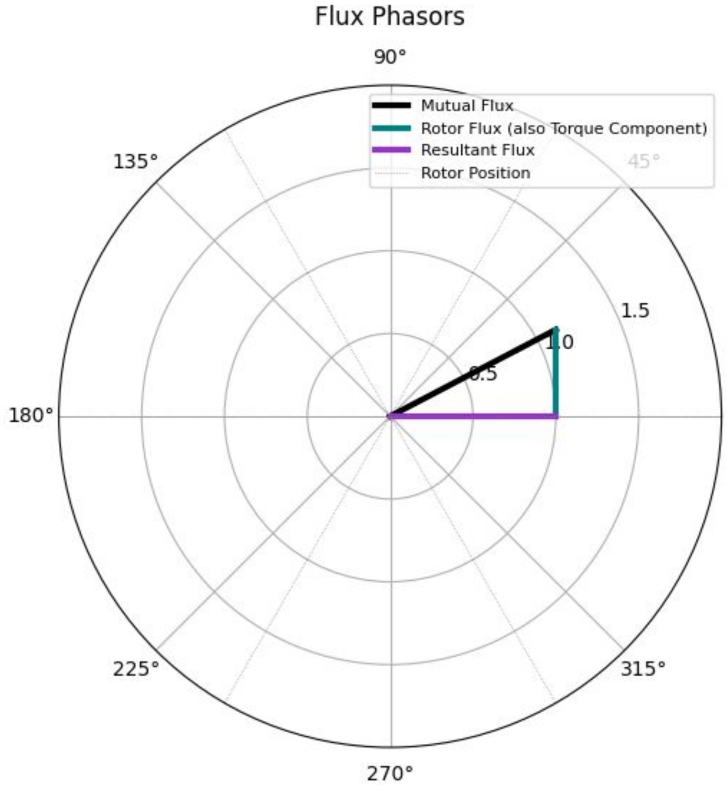


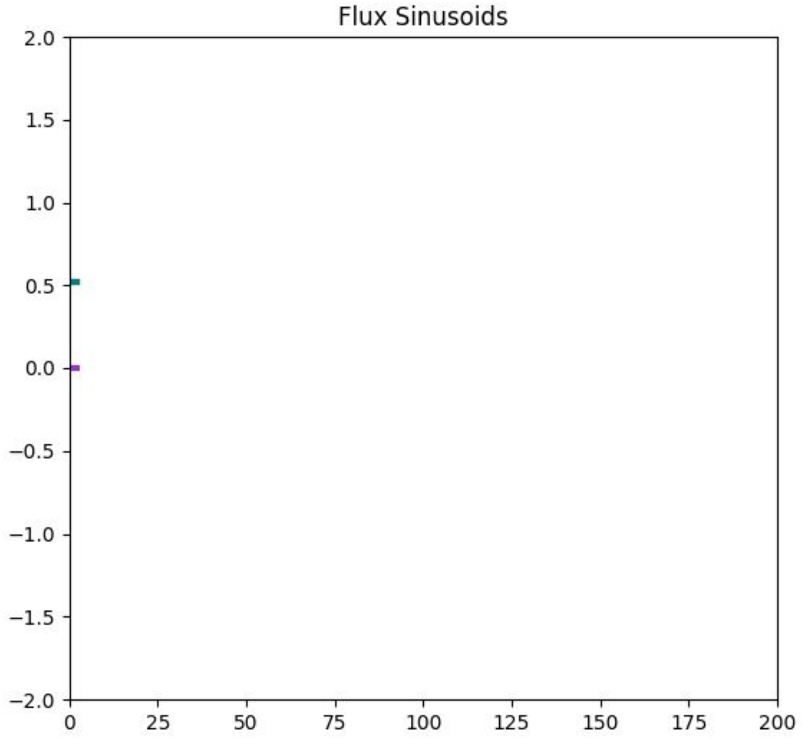


Instantaneous Thrust Control (FOC)

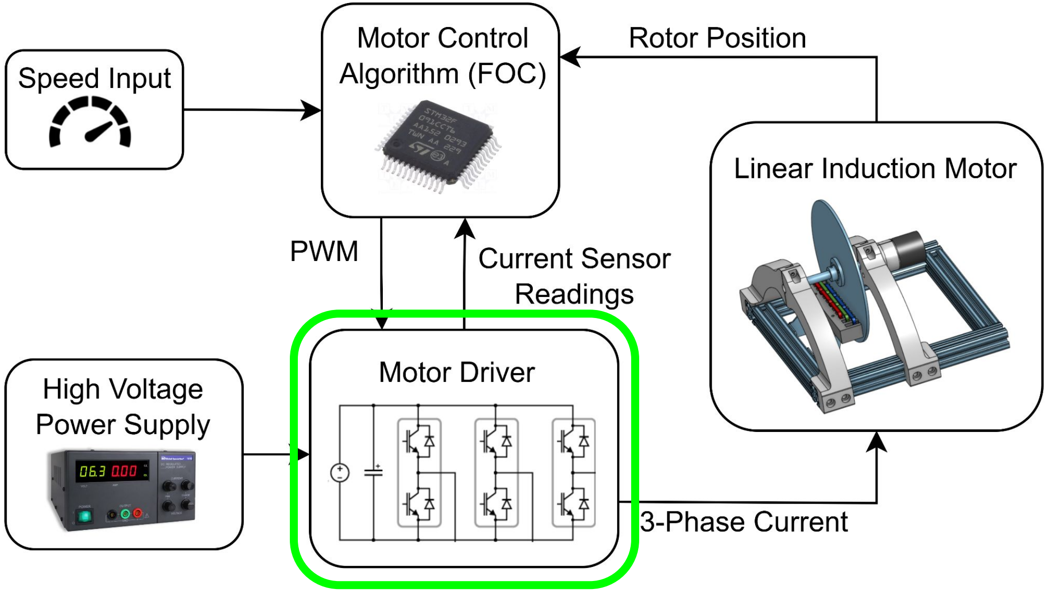


Instantaneous Thrust Control (FOC)





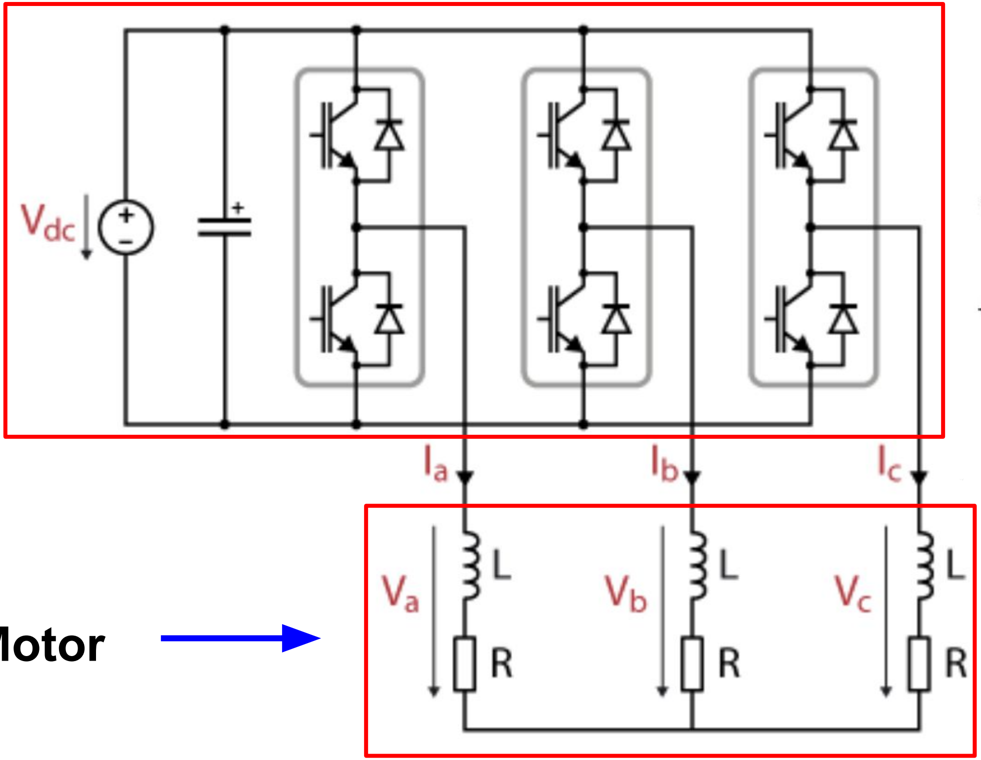
Motor Driver and Gate Driver Design

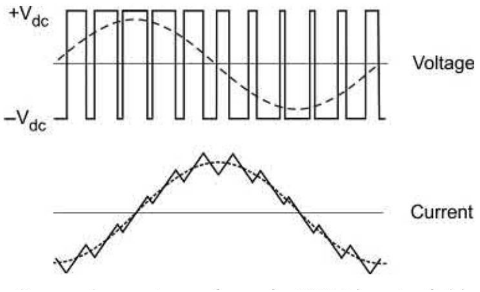


3-Phase Inverter

3-Phase Inverter

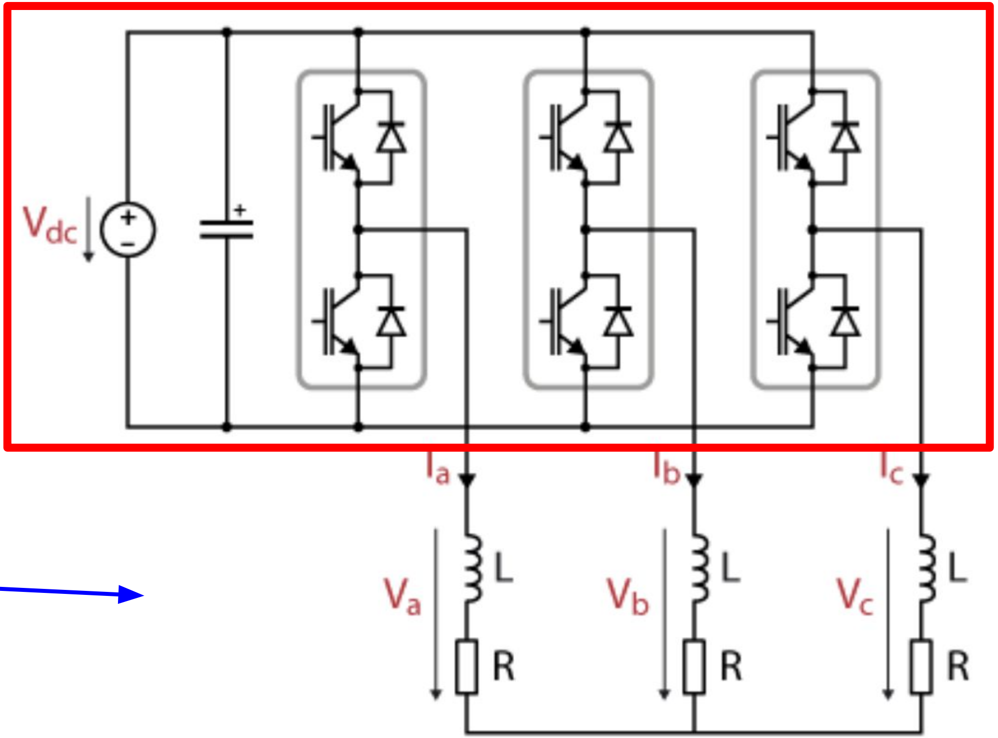
Motor





Source for image

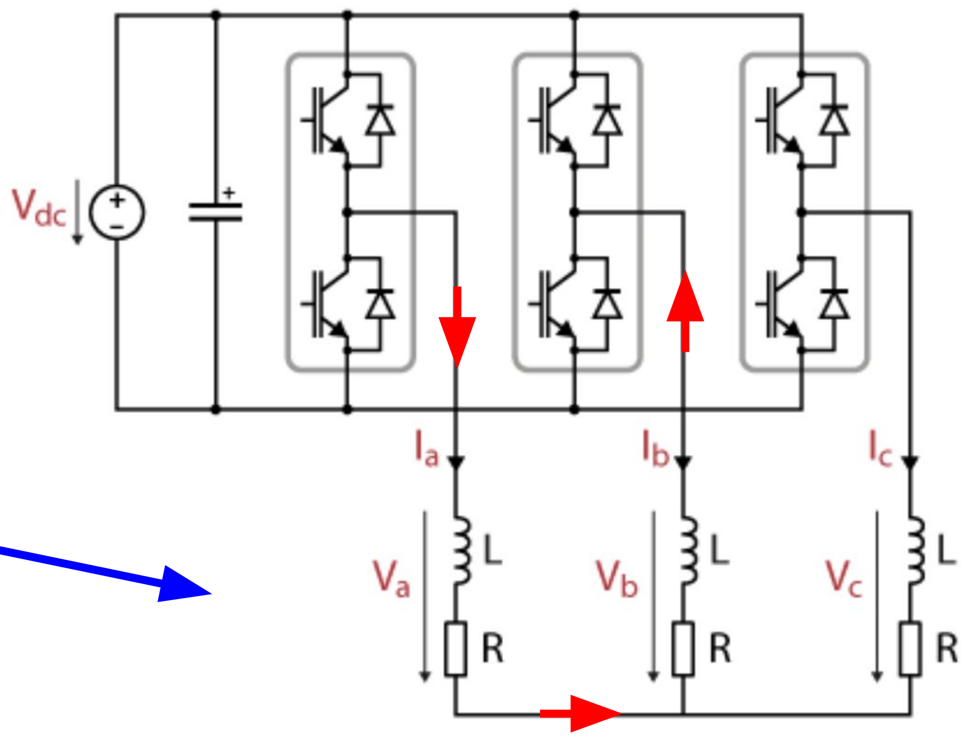
3-Phase Inverter



DC Input is converted into 3-Phase AC for the motor

Source for image

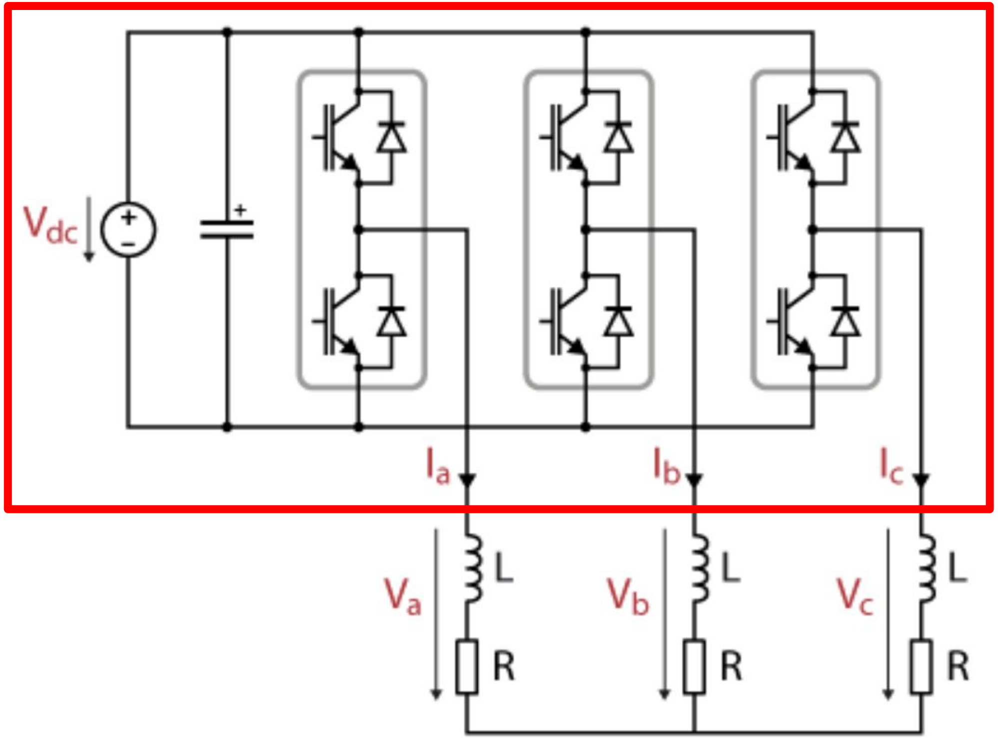
3-Phase Inverter (with motor)



Star config allows current to go through **both** directions

Source for image

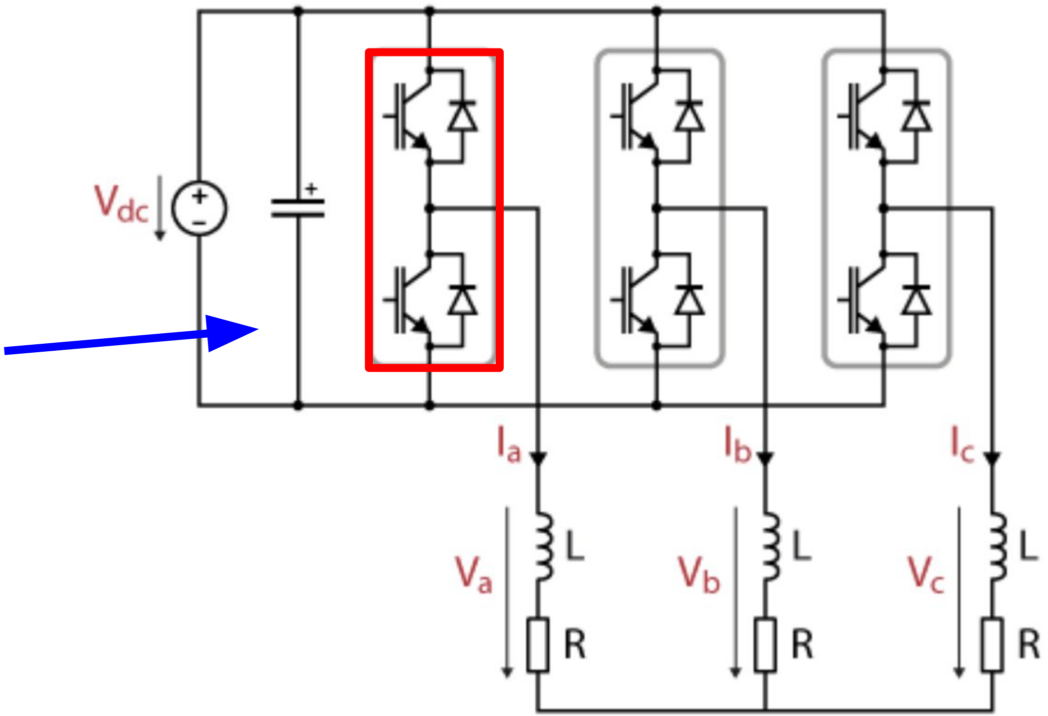
3-Phase Inverter



Source for image

Single Half-bridge of 3-Phase inverter

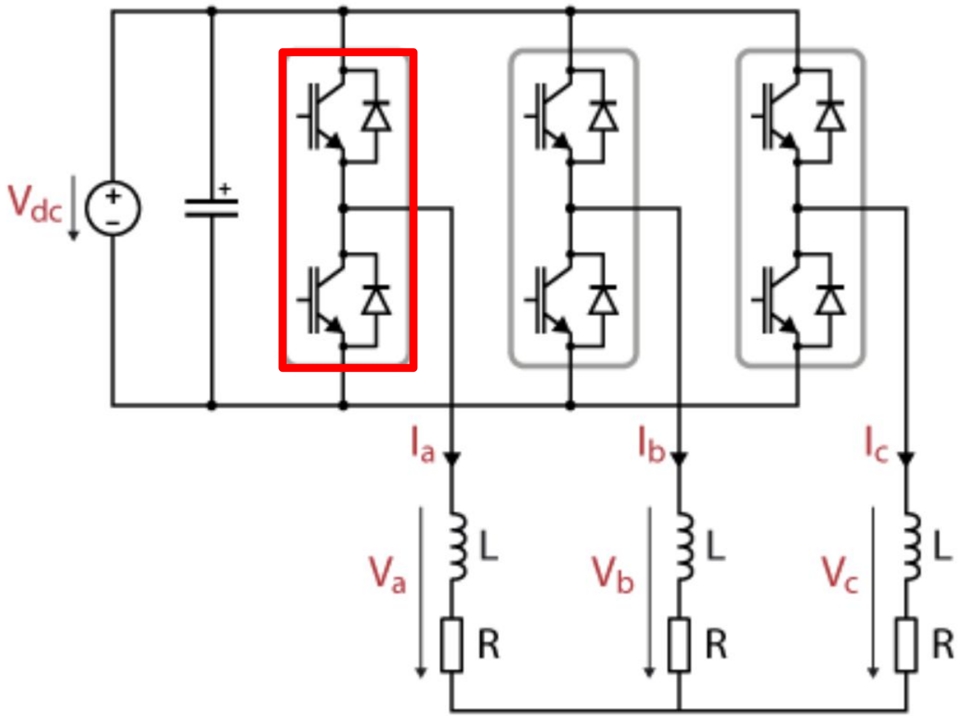
Need Half-bridge to alter polarity of the circuit



Source for image

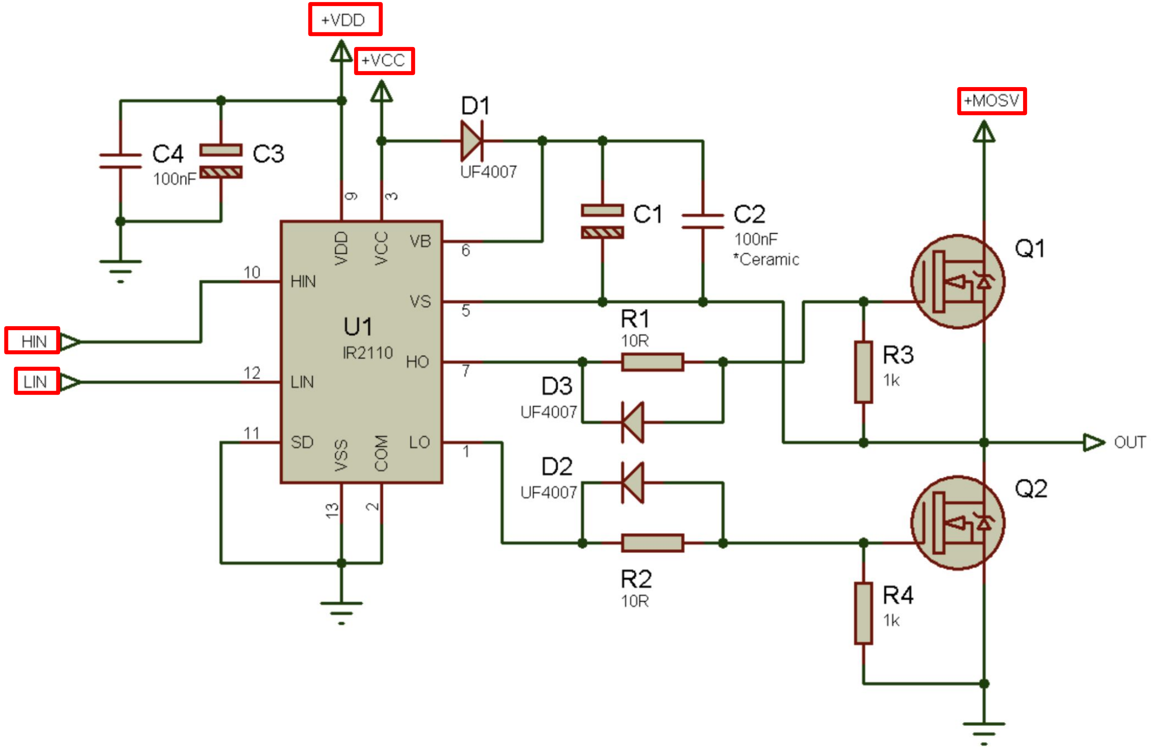
Single Half-bridge of 3-Phase inverter

Also need IC gate driver in Half-bridge!



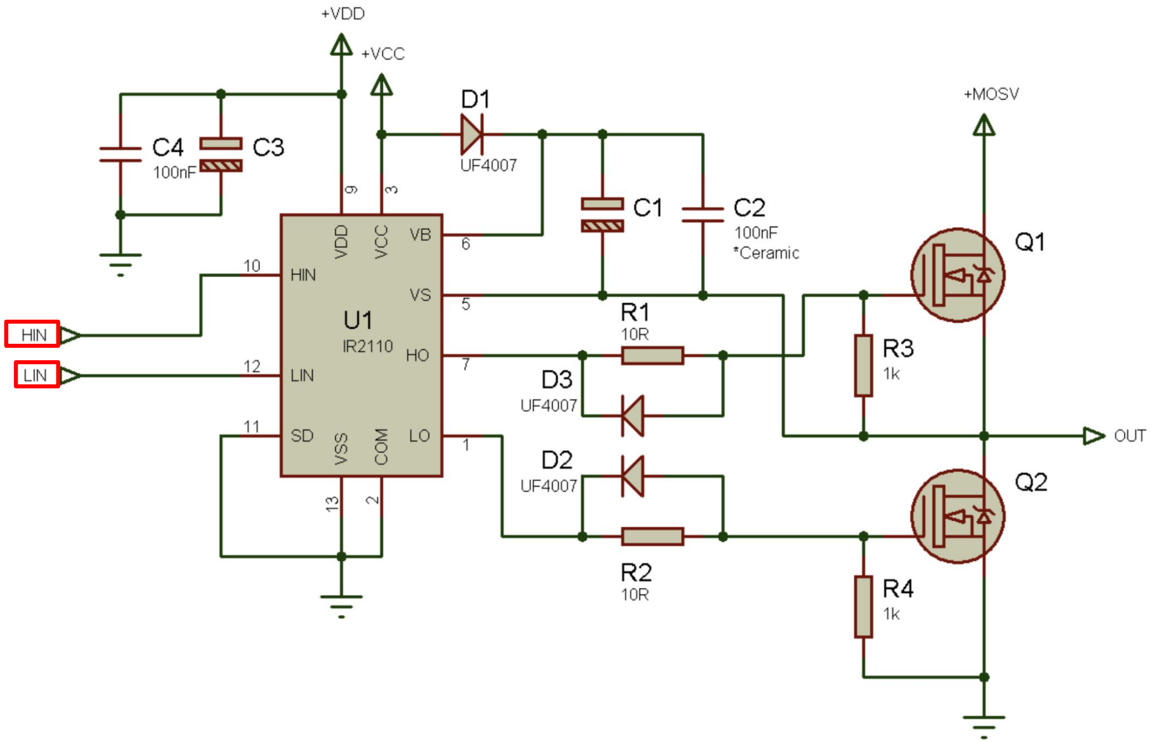
Source for image

Single Half-Bridge with gate driver



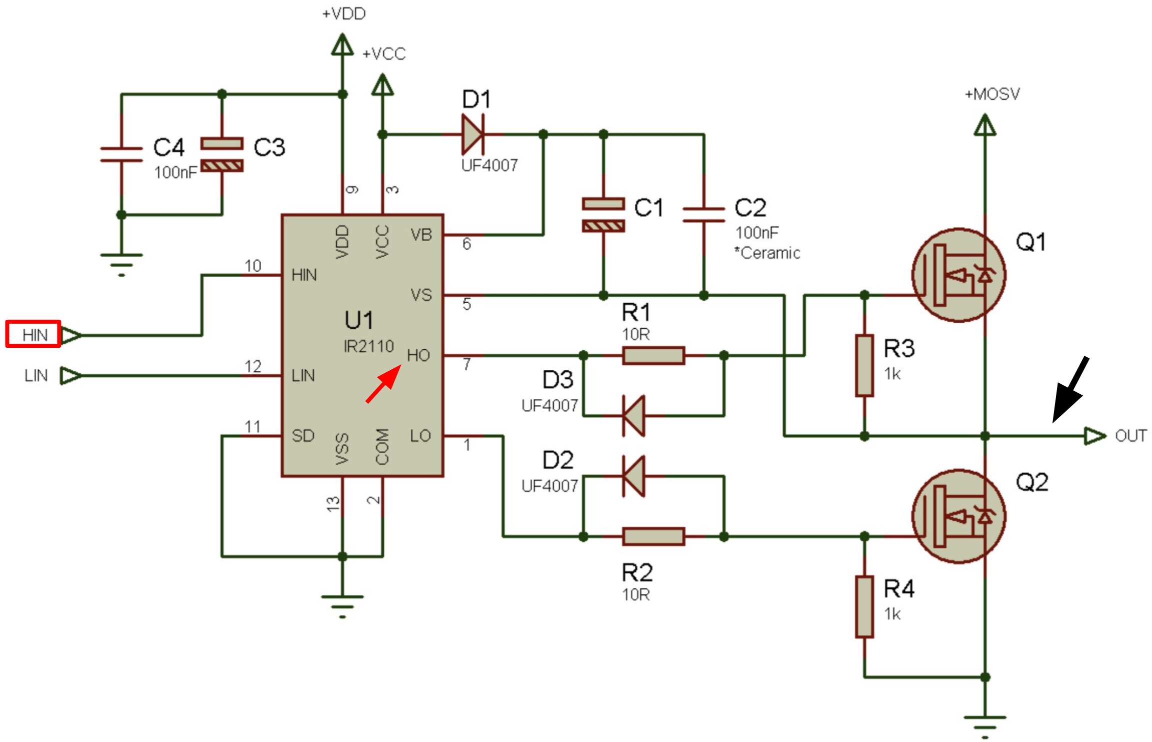
Source for image

Logic of single Half-Bridge with gate driver



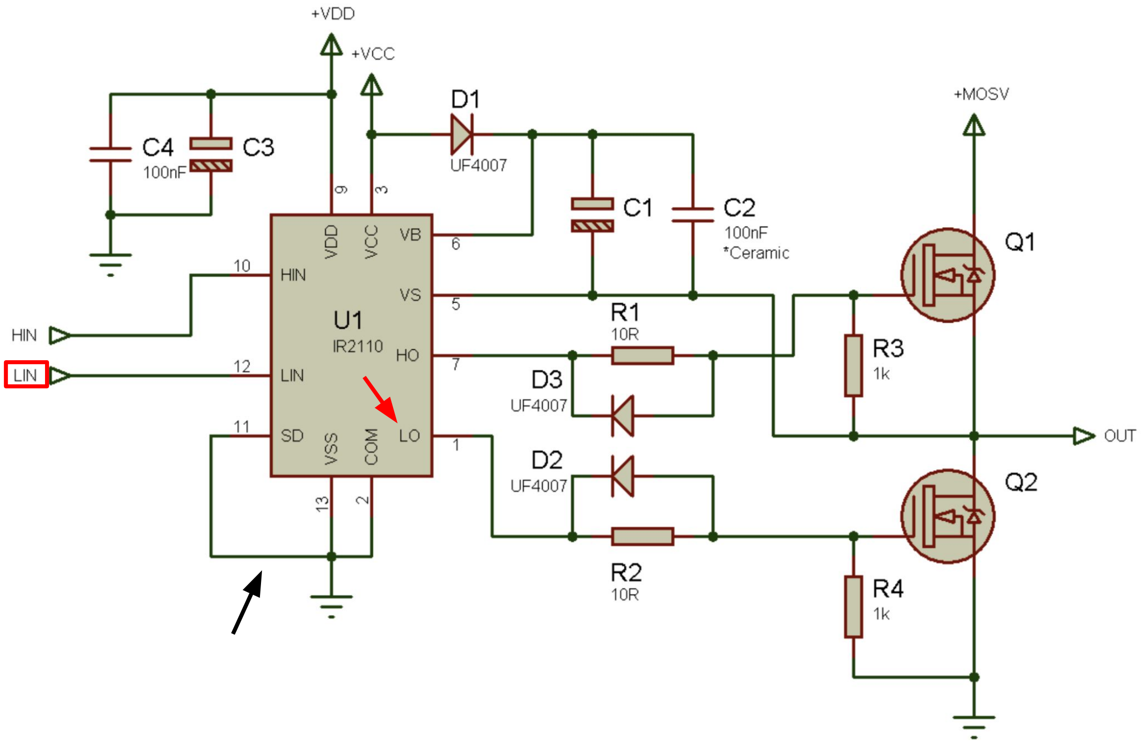
Logic
Inputs

Logic of single Half-Bridge with gate driver (HIN)



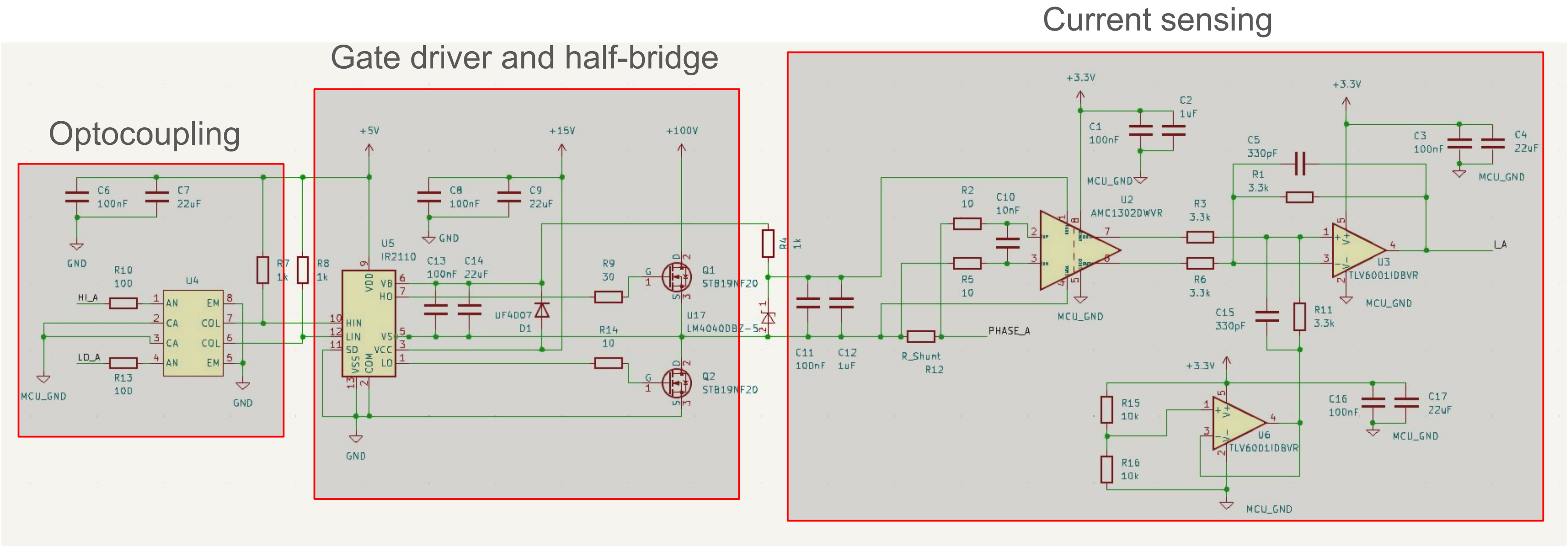
Source for image

Logic of single Half-Bridge with gate driver (LIN)

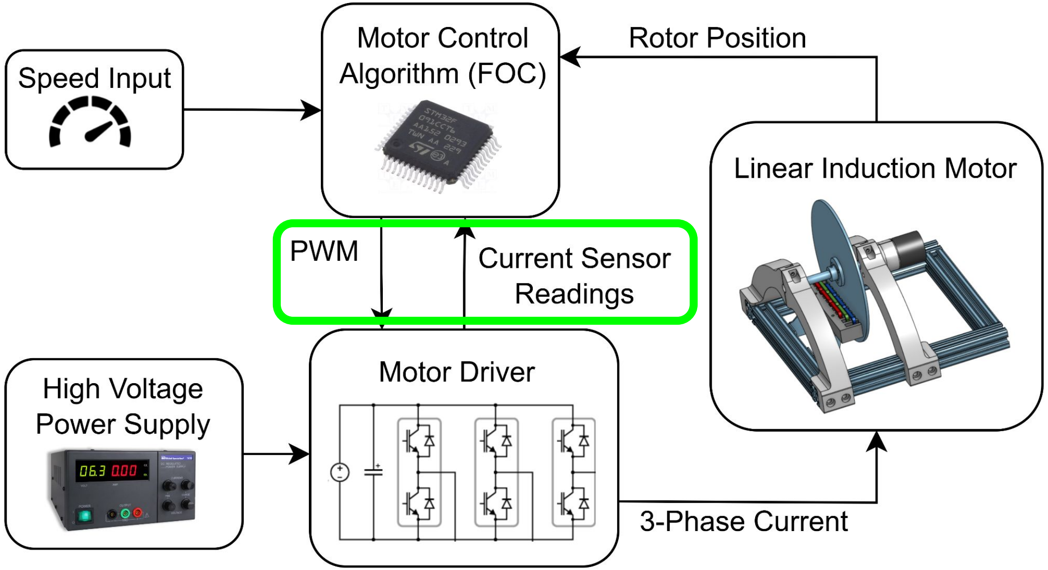


Source for image

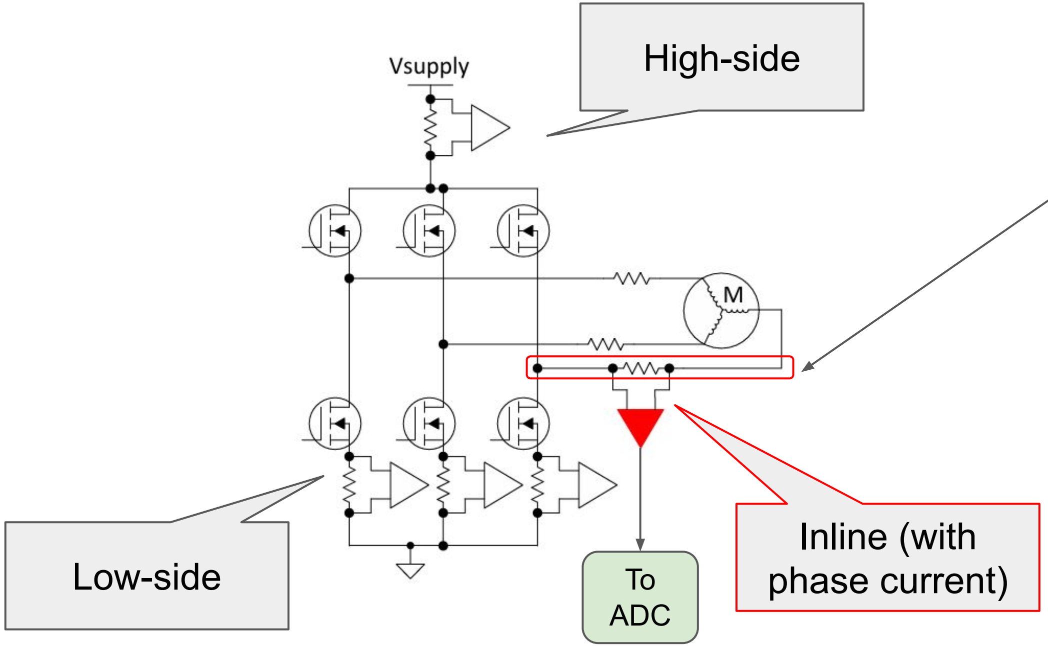
Single Half-Bridge with Current Sensor



Current sensing and PWM output with STM32 microprocessor



3-Phase current sensing: Inline is simplest!



This node switches between 0 and 110V!

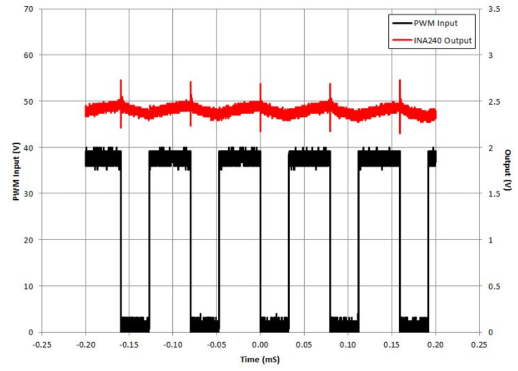


Figure 8. INA240 Output vs PWM Input

TI: Current sensing for inline motor-control applications

Op-amp must reject Common-Mode PWM signal!

INA240 is pretty stable

The big dip :(

Very large spikes :(

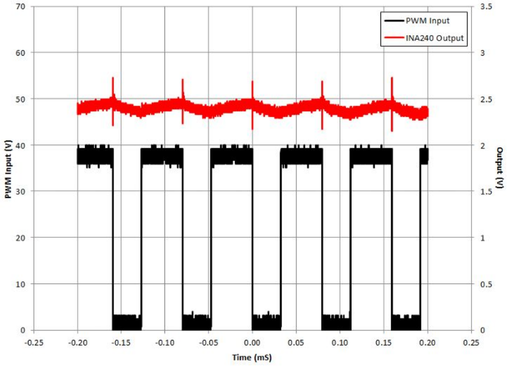


Figure 8. INA240 Output vs PWM Input

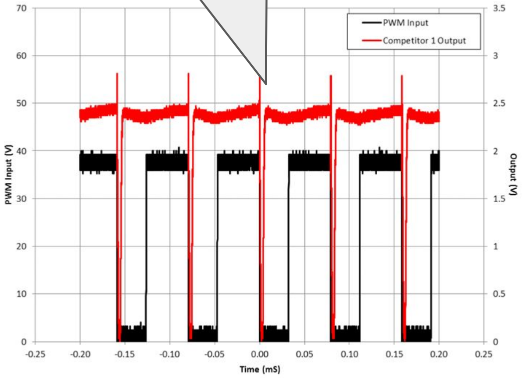


Figure 9. Competitor 1 Output vs PWM Input

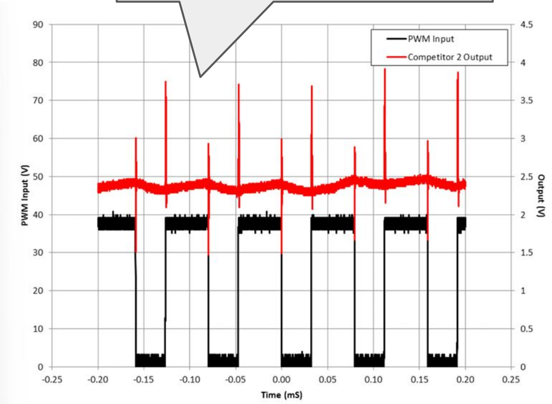
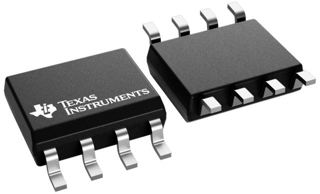


Figure 10. Competitor 2 Output vs PWM Input

PWM Square wave,
50us on / 30us off (25 kHz)



Op-amp must reject Common-Mode PWM signal!

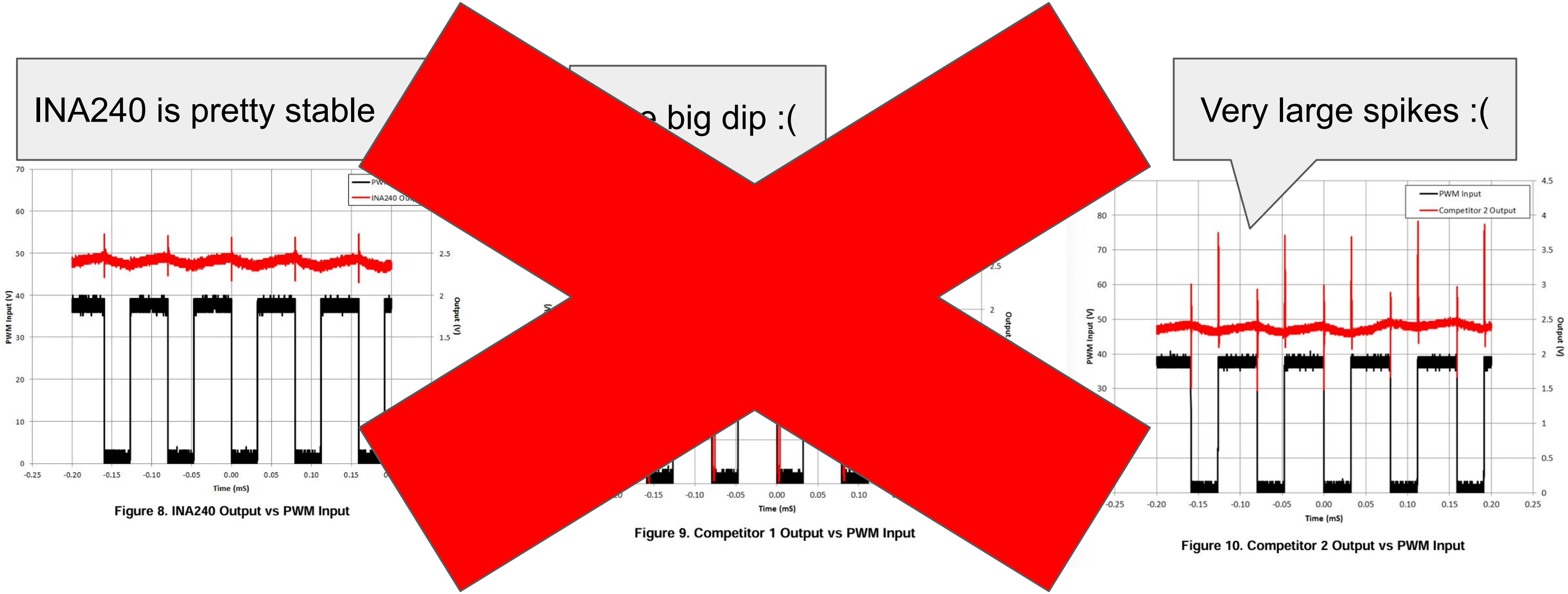
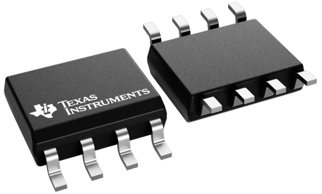


Figure 8. INA240 Output vs PWM Input

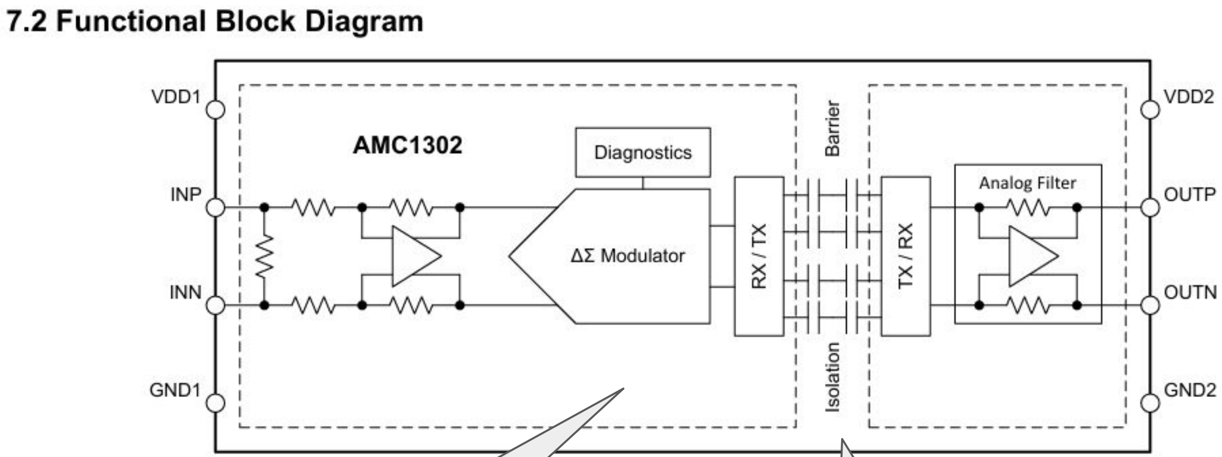
Figure 9. Competitor 1 Output vs PWM Input

Figure 10. Competitor 2 Output vs PWM Input

PWM Square wave,
50us on / 30us off (25 kHz)



Use Opto-Isolated amplifiers instead?



Digital signal;
LED coupling!
Common-mode voltage
is never seen

Isolation
barrier up to
6000V!

How to use an opto-isolated opamp (AMC1302 datasheet)

Supply & Inputs must be less than 6V!
Use floating GND1 and supply.

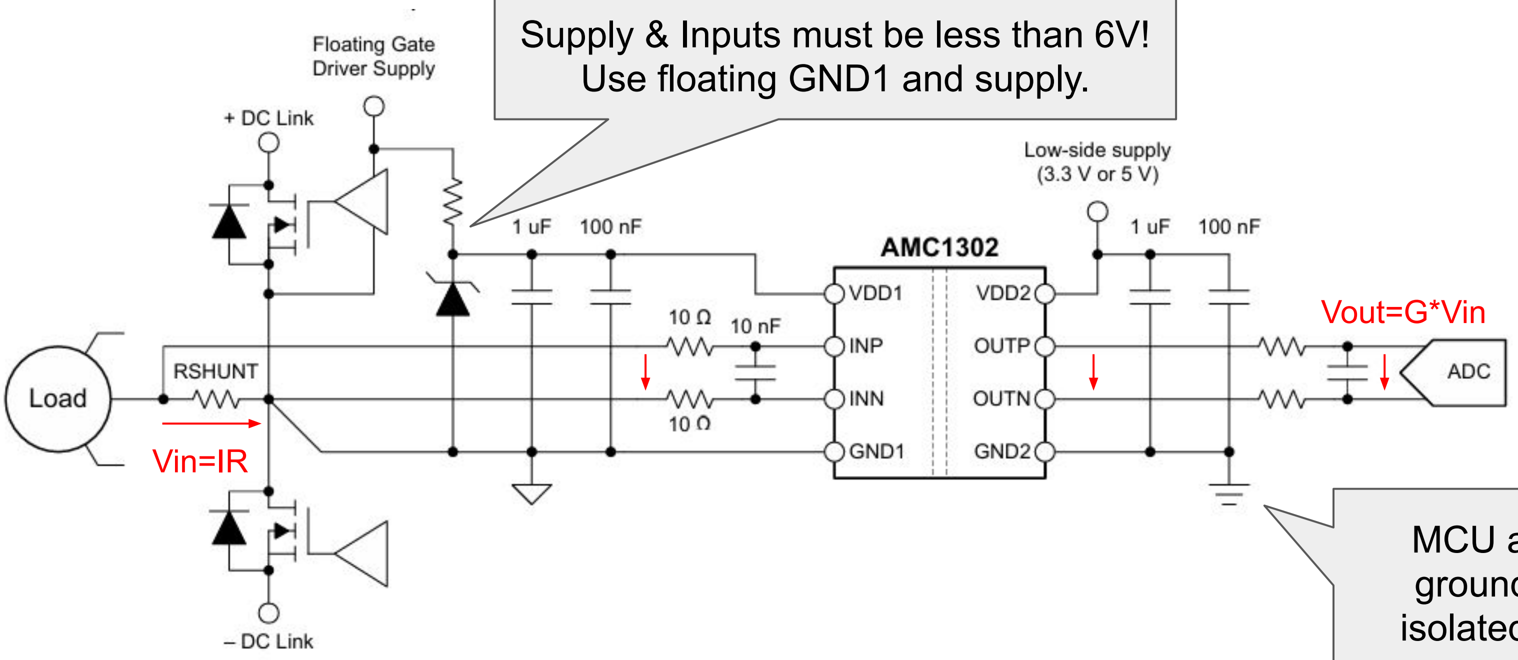


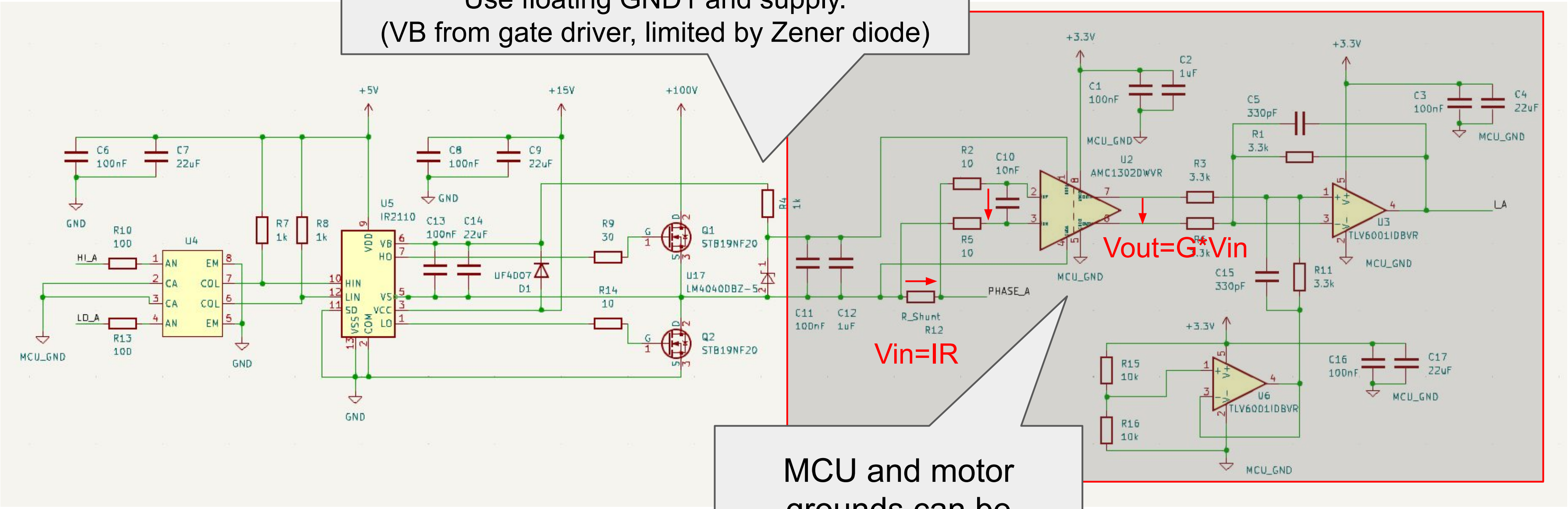
Figure 8-1. Using the AMC1302 for Current Sensing in a Typical Application

MCU and motor grounds can be isolated, blocking ground noise

How to use an opto-isolated opamp (our PCB)

Supply & Inputs must be less than 6V!
Use floating GND1 and supply.
(VB from gate driver, limited by Zener diode)

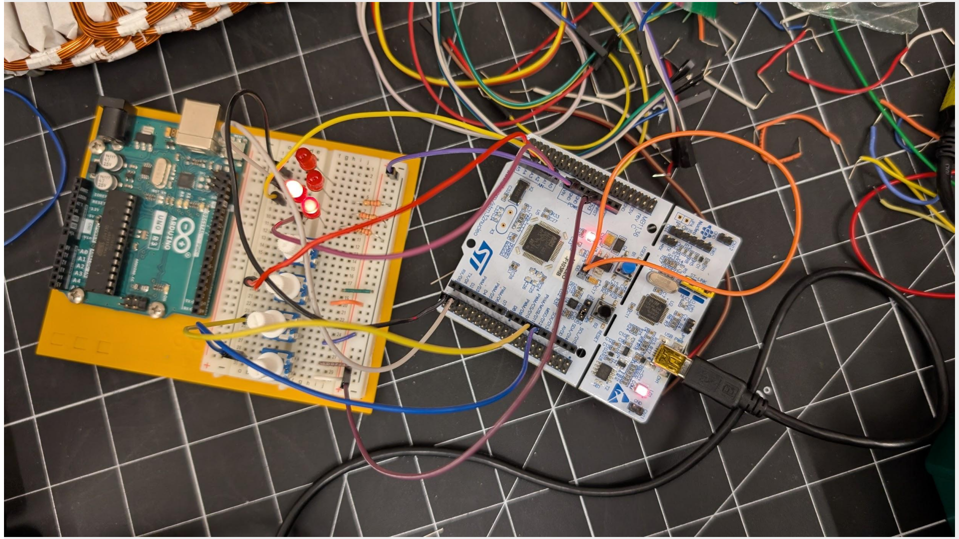
Current sensing

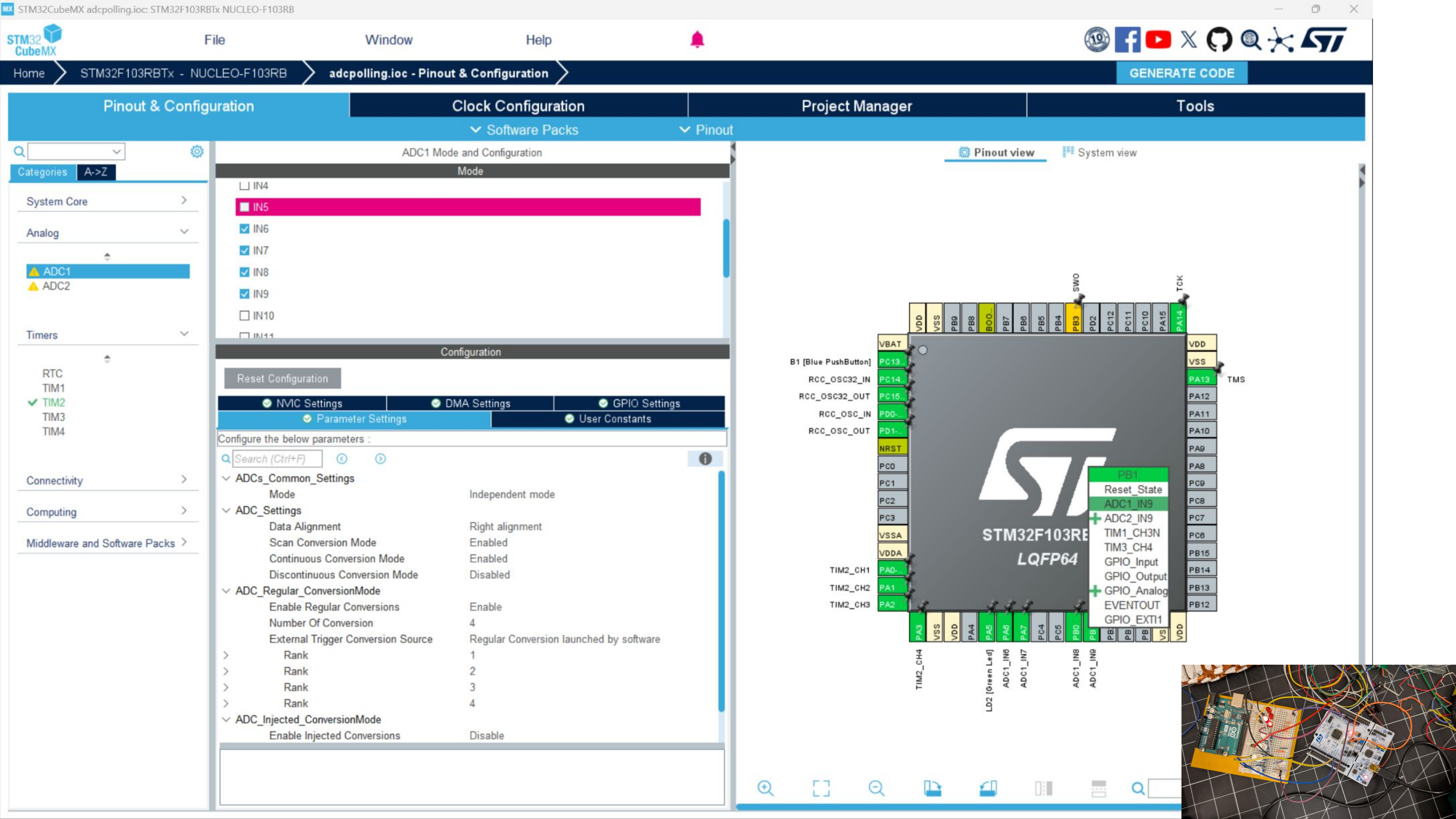


MCU and motor grounds can be isolated, blocking ground noise

STM32 DMA Demo!

4-LED Dimmer with analog potentiometer control and 1kHz PWM output



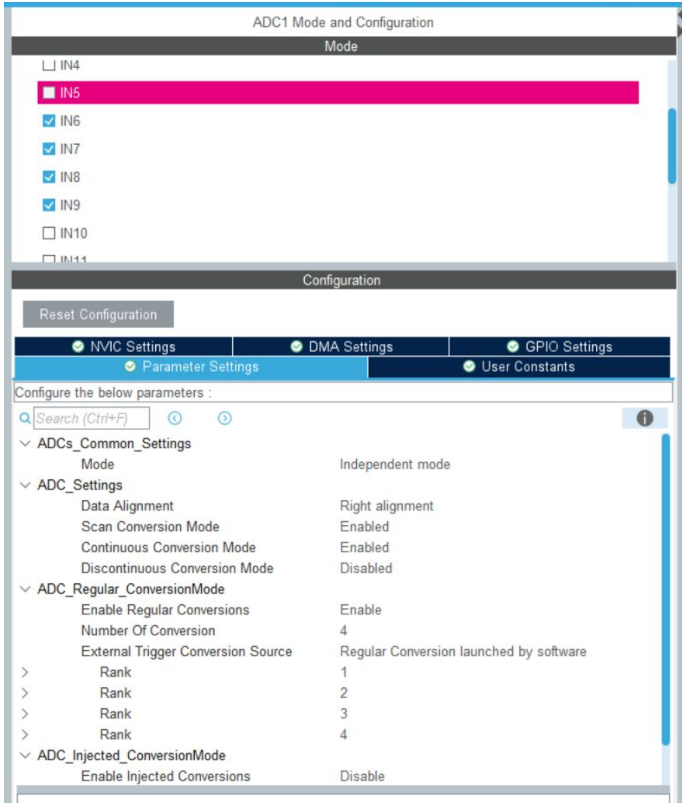

STM32CubeMX adcpolling.ioc: STM32F103RBTx NUCLEO-F103RB
File
Window
Help
Home
STM32F103RBTx - NUCLEO-F103RB
adcpolling.ioc - Pinout & Configuration
GENERATE CODE
Pinout & Configuration
Clock Configuration
Project Manager
Tools
Software Packs
Pinout
Categories
A->Z
System Core
Analog
ADC1
ADC2
Timers
RTC
TIM1
TIM2
TIM3
TIM4
Connectivity
Computing
Middleware and Software Packs
ADC1 Mode and Configuration
Mode
IN4
IN5
IN6
IN7
IN8
IN9
IN10
Configuration
Reset Configuration
NVIC Settings
DMA Settings
GPIO Settings
Parameter Settings
User Constants
Configure the below parameters :
Search (Ctrl+F)
ADCs_Common_Settings
Mode Independent mode
ADC_Settings
Data Alignment Right alignment
Scan Conversion Mode Enabled
Continuous Conversion Mode Enabled
Discontinuous Conversion Mode Disabled
ADC_Regular_ConversionMode
Enable Regular Conversions Enable
Number Of Conversion 4
External Trigger Conversion Source Regular Conversion launched by software
Rank 1
Rank 2
Rank 3
Rank 4
ADC_Injected_ConversionMode
Enable Injected Conversions Disable
Pinout view
System view
STM32F103RB
LQFP64
B1 [Blue PushButton]
RCC_OSC32_IN
RCC_OSC32_OUT
RCC_OSC_IN
RCC_OSC_OUT
TIM2_CH1
TIM2_CH2
TIM2_CH3
TMS
SWO
TCK
TIM2_CH4
LD2 [Green Led]
ADC1_IN6
ADC1_IN7
ADC1_IN8
ADC1_IN9
PB1
Reset_State
ADC1_IN9
ADC2_IN9
TIM1_CH3N
TIM3_CH4
GPIO_Input
GPIO_Output
GPIO_Analog
EVENTOUT
GPIO_EXTI1

Use Direct Memory Access (DMA) to read ADC values!

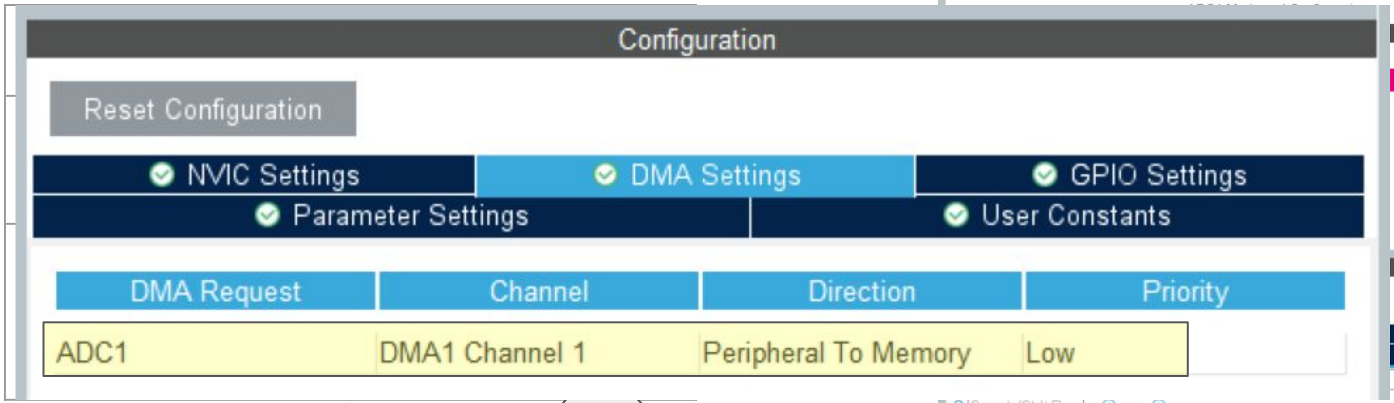
Analog-to-Digital converter modes	
Single-Channel Single-conversion	Multi-channel Single-conversion
Single-Channel Continuous-conversion	<u>Multi-channel Continuous-conversion</u>

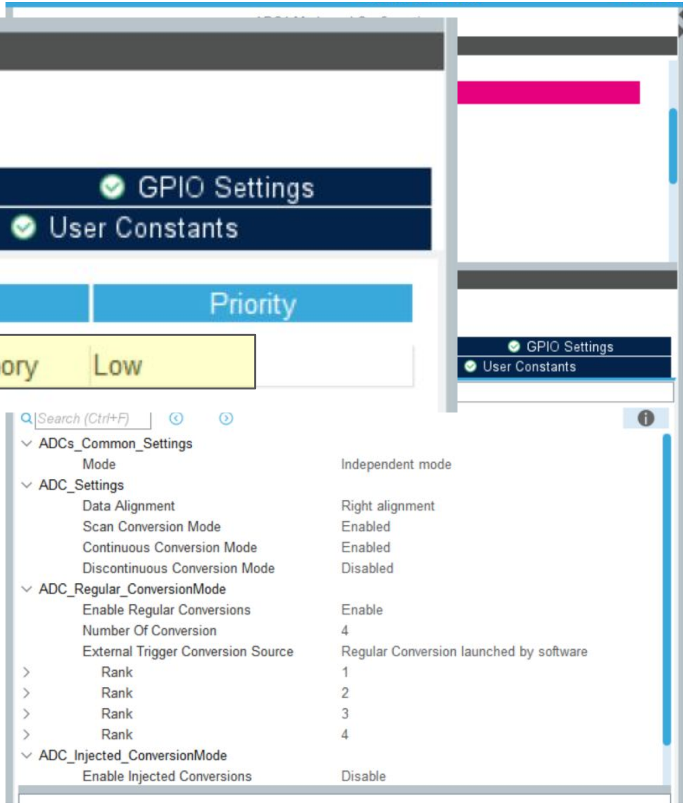
I can't poll or use CPU interrupts if the ADC produces 1.1M samples/second.

I need a separate memory controller to move the 12-bit ADC readings from register to RAM!



Use Direct Memory Access (DMA) to read ADC values!

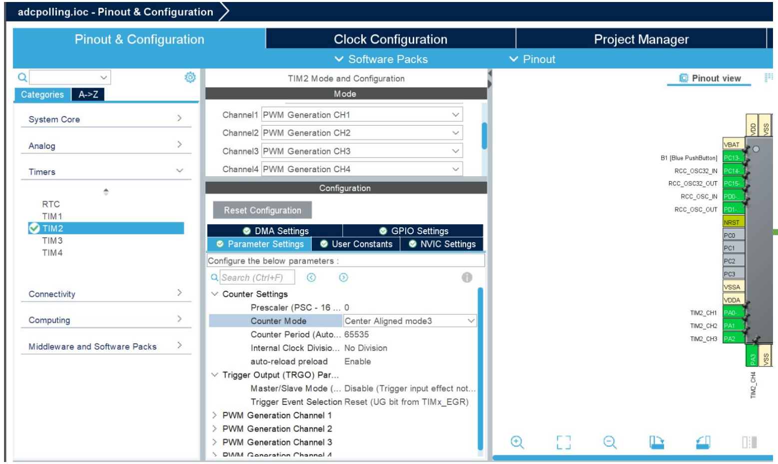


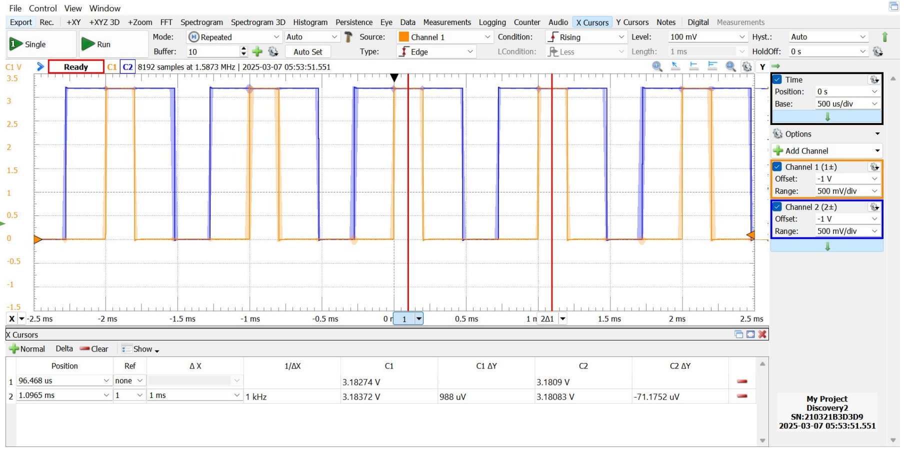


I can't poll or use CPU interrupts if the ADC produces 1.1M samples/second.

I need a separate memory controller to move the 12-bit ADC readings from register to RAM!

Define PWM Output with timers!





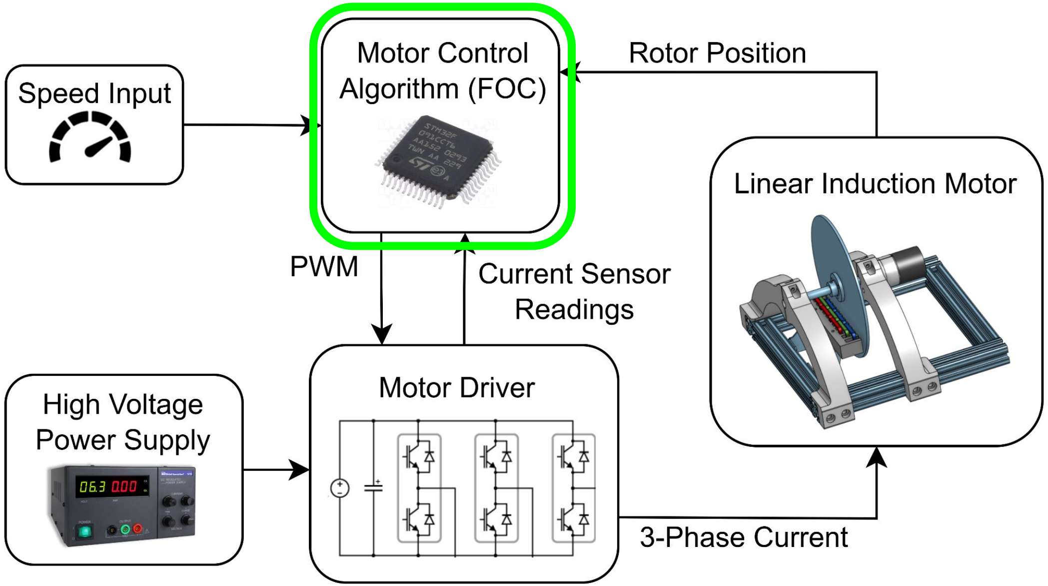
$$F_{PWM} = \frac{F_{CLK}[72\text{MHz}]}{(ARR+1)(PSC[0..65535]+1)}$$

$$\text{Duty Cycle} = \frac{CCR[0..65535]}{ARR[0..65535]} \times 100\%$$

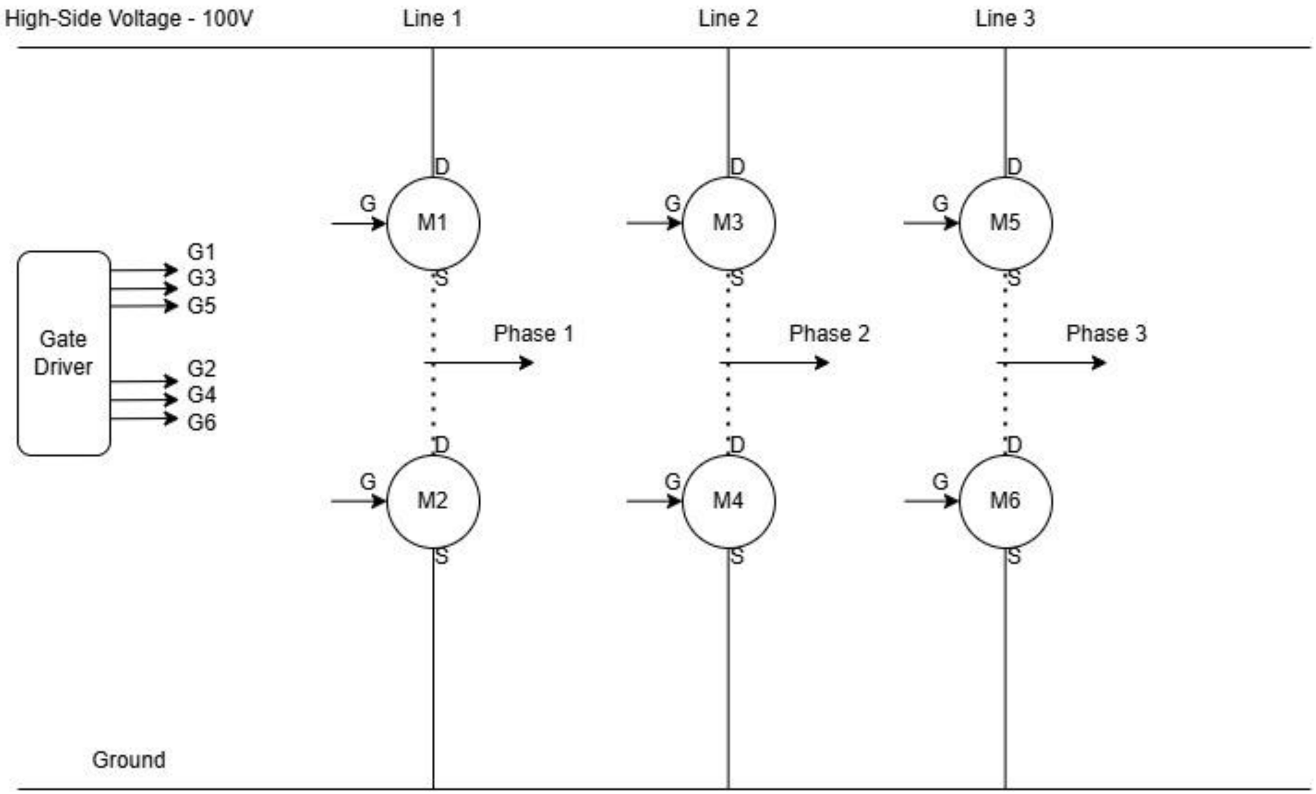
For center-aligned PWM, use **half** the value of ARR+1 from this formula!

Driving 3-phase inverter with centered pulses **reduces switching losses and EMI?**

Software:
Gate Driver
Field-Oriented Control



Step 1 - Measure Real-time 3-Phase Current



Step 2 - Clarke Transform

In Step 1, we measured the 3-phase current values. We will call them i_a, i_b, and i_c. We use the Clarke Transform to convert the 3-phase current to a 2-D stationary reference frame.

$$
\begin{aligned}
i_{\alpha\beta}(t) &= \frac{2}{3}\begin{bmatrix} 1 & -\frac{1}{2} & -\frac{1}{2} \\ 0 & \frac{\sqrt{3}}{2} & -\frac{\sqrt{3}}{2} \end{bmatrix}\begin{bmatrix} i_a(t) \\ i_b(t) \\ i_c(t) \end{bmatrix} \\
&= \begin{bmatrix} 1 & 0 \\ \frac{1}{\sqrt{3}} & \frac{2}{\sqrt{3}} \end{bmatrix}\begin{bmatrix} i_a(t) \\ i_b(t) \end{bmatrix}
\end{aligned}
$$

Step 3 - Park Transform

Using an encoder built into our setup, we can measure the real-time rotor angle of the motor. Using this angle, θ_{rotor}, i_α, and i_β computed in Step 2, we perform a Park Transform to convert our current values from the stationary reference frame to the rotor aligned DQ-frame (Direct-Quadrature frame).

Park Transform - General Case:

$$\vec{v}_{DQ} = \begin{bmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{bmatrix} \cdot \vec{v}_{XY}\,.$$

$$i_d = i_\alpha cos(\theta_{rotor}) + i_\beta sin(\theta_{rotor})$$
$$i_q = -i_\alpha sin(\theta_{rotor}) + i_\beta cos(\theta_{rotor})$$

Direct axis component is aligned with rotor flux while the Quadrature axis component is responsible for torque.

Step 4 - PI-Control

We are only interested in controlling the torque of the motor and not the rotor flux.
The set point value for the direct axis component, $i_{d,\mathrm{ref}}$, is zero.
The set-point value for the quadrature axis component, $i_{q,\mathrm{ref}}$, is the current required to meet the torque demand from the motor.

$$i_d^+ = K_{P_d}(i_{d,\mathrm{ref}} - i_d) + K_{I_d}\int(i_{d,\mathrm{ref}} - i_d)\,dt$$
$$i_q^+ = K_{P_q}(i_{q,\mathrm{ref}} - i_q) + K_{I_q}\int(i_{q,\mathrm{ref}} - i_q)\,dt$$

Step 5 - Inverse Park Transform

Now that we have computed the desired controlled current in the DQ-Frame, we use the inverse Park Transform to convert them to the stationary 2-D reference frame. This is necessary for the final step where we compute the gate driver input for this field-oriented controller.

$$i_\alpha^+ = i_d^+ cos(\theta_{rotor}) - i_q^+ sin(\theta_{rotor})$$
$$i_\beta^+ = i_d^+ sin(\theta_{rotor}) + i_q^+ cos(\theta_{rotor})$$

Step 6 - Space Vector Pulse-Width Modulation (SVPWM)

The i_α^+ and i_β^+ values computed in Step 5 form a reference vector $\vec{i_{\text{ref}}} = i_\alpha^+ + \mathbf{j} i_\beta^+$. There are six possible active states and two corresponding zero states. The state is dependent on the sector in which $\vec{i_{\text{ref}}}$ lies.

$$i_{\text{ref}} = \sqrt{(i_\alpha^+)^2 + (i_\beta^+)^2}$$

$$\theta = arctan\left[\frac{i_\beta^+}{i_\alpha^+}\right]$$

$$\text{Sector Number - } SN = \left\lfloor \frac{\theta}{\frac{\pi}{3}} \right\rfloor$$

$$\theta_s = \theta - (SN - 1)\frac{\pi}{3}$$

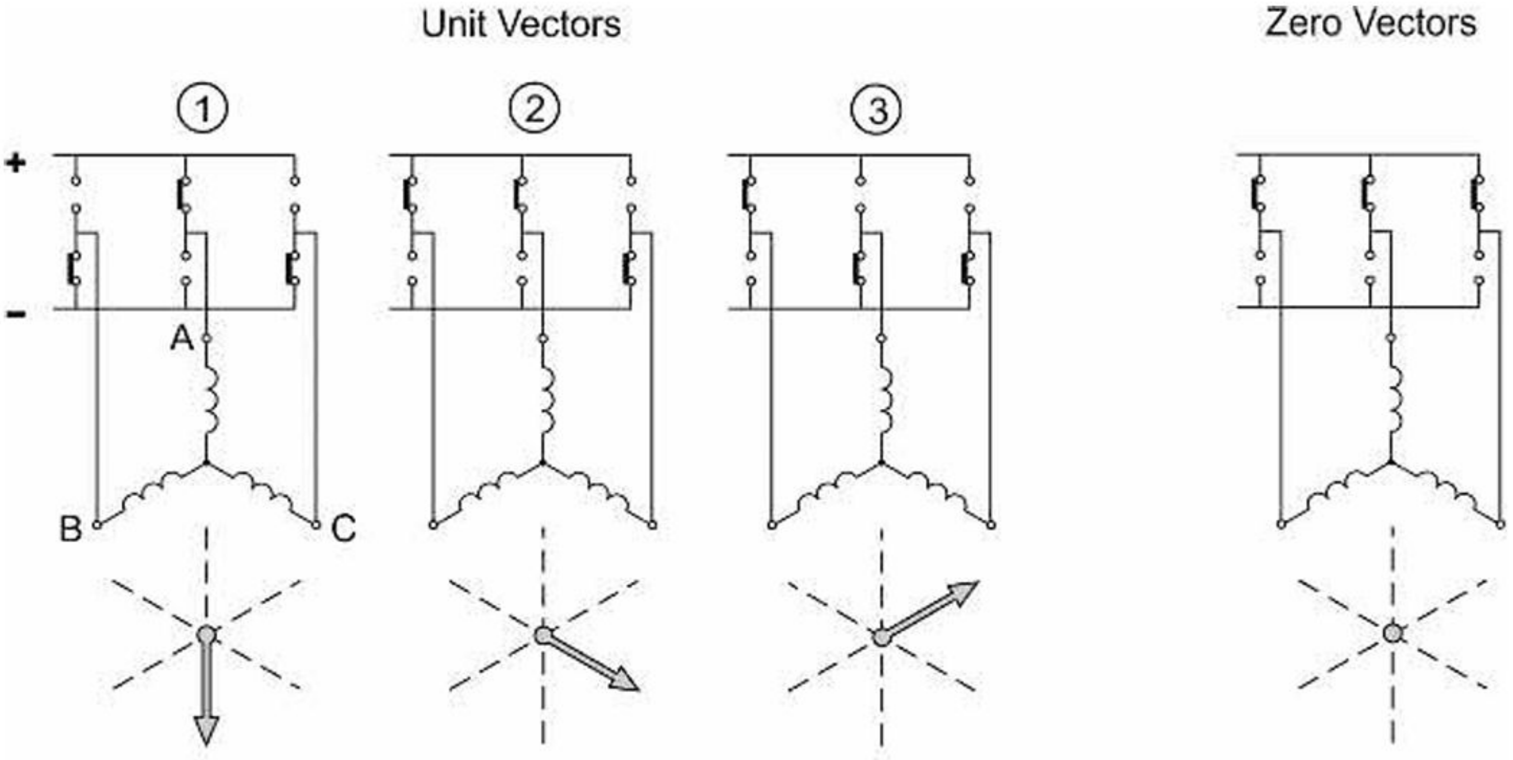
$$T_1 = T_s \frac{i_{\text{ref}}}{i_{DC}} sin(\frac{\pi}{3} - \theta_s)$$

$$T_2 = T_s \frac{i_{\text{ref}}}{i_{DC}} sin(\theta_s)$$

$$T_0 = T_s - (T_1 + T_2)$$

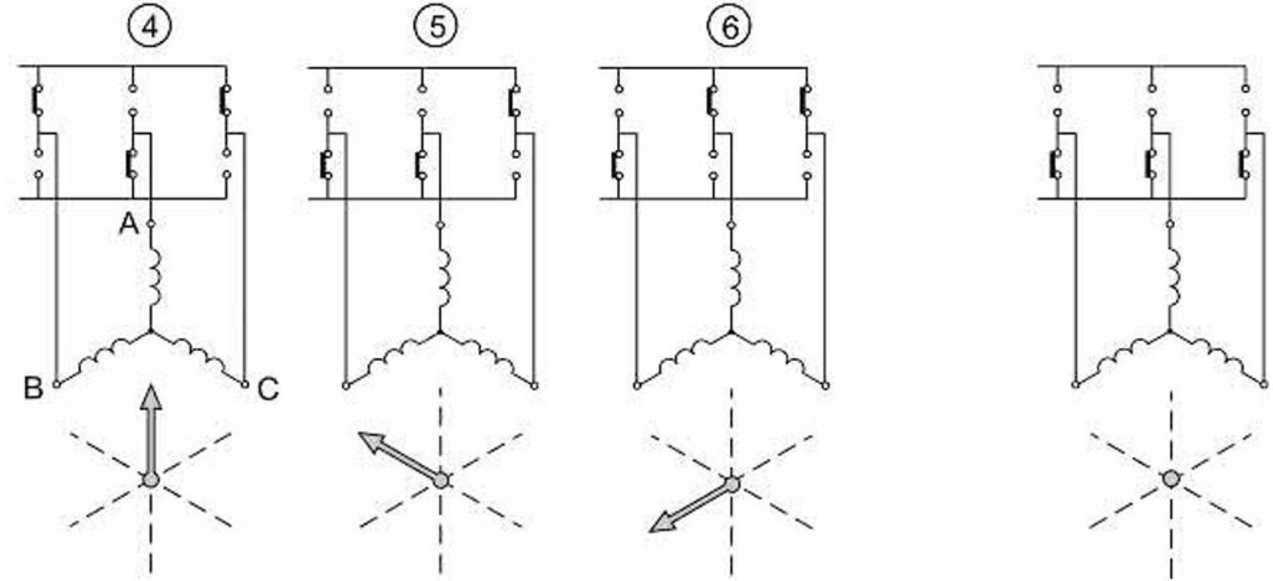
T_s is the PWM switching period which is determined by the PWM frequency.

Step 6 - Continued



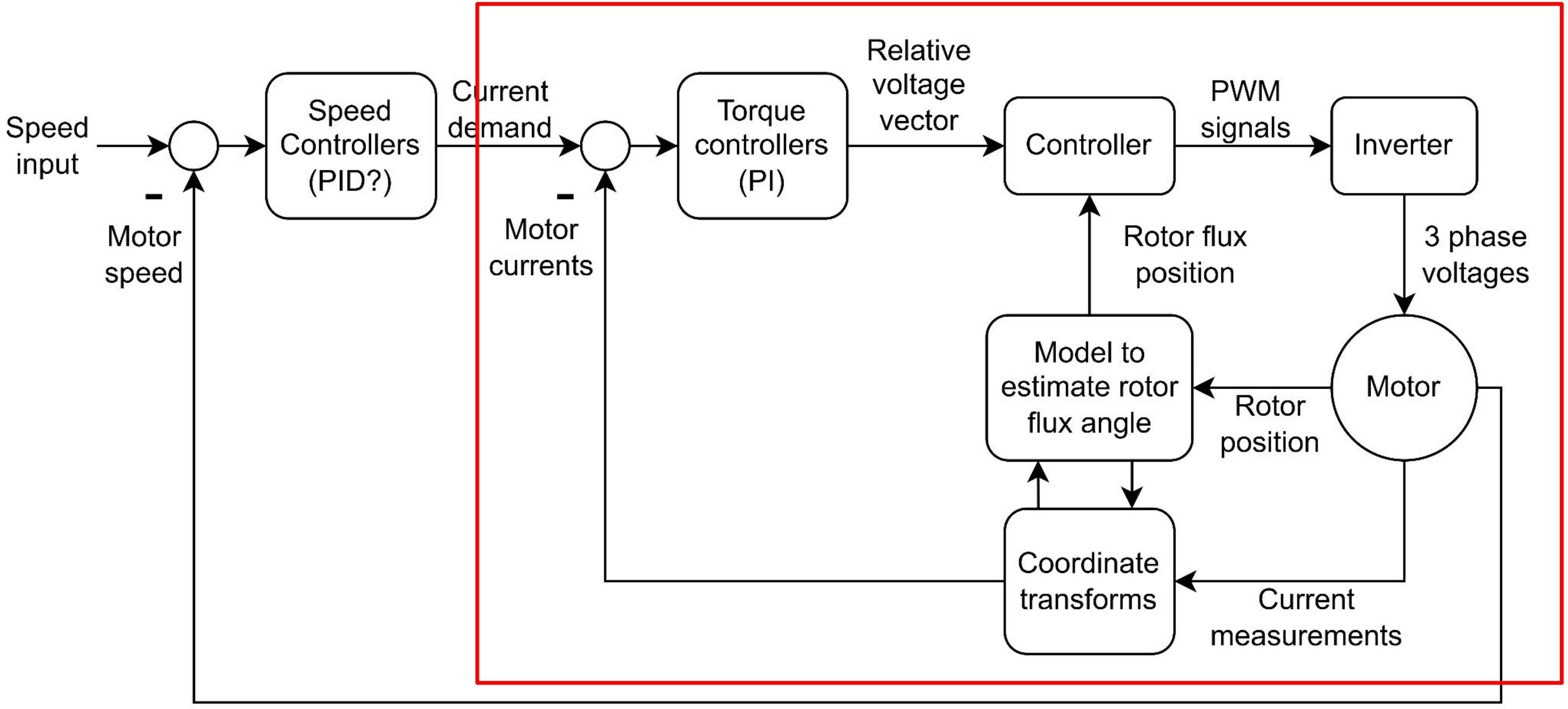
From Drury & Hughes - Electric Motors and Drives

Step 6 - Continued



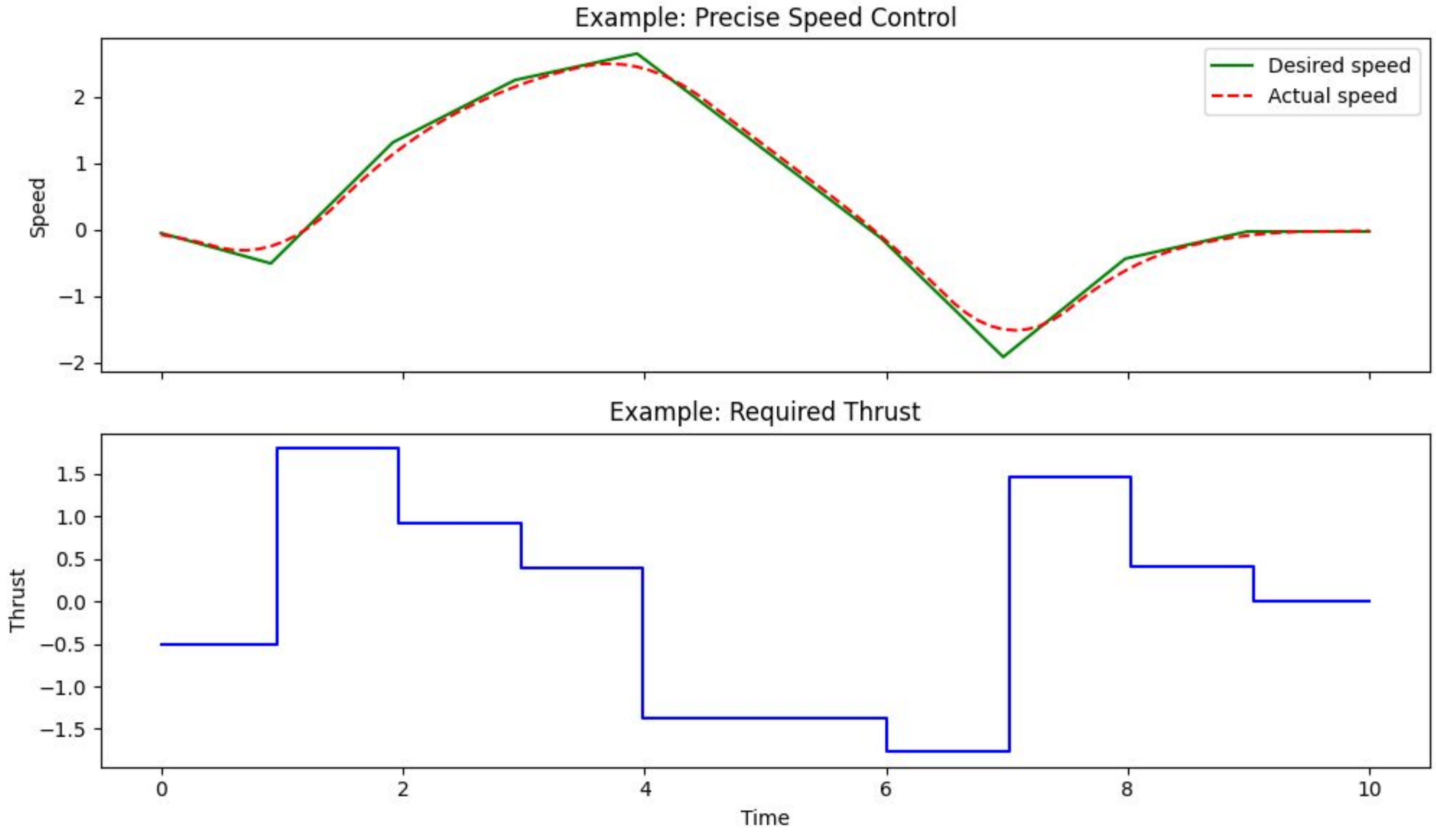
For example, if $SN = 1$, we drive the gates in the Active 1 configuration for a period of T_1, and then in the Active 2 configuration for a period of T_2. Finally, we drive the gates in the corresponding Zero configuration for a period of T_0.

Control loop diagram

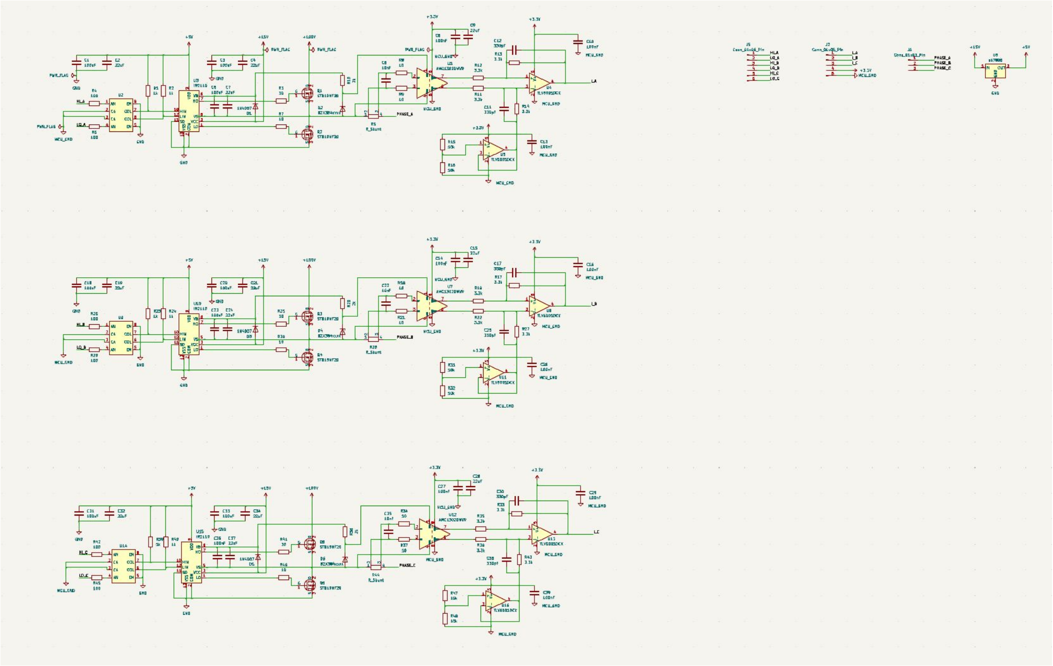


Deliverables and End Result

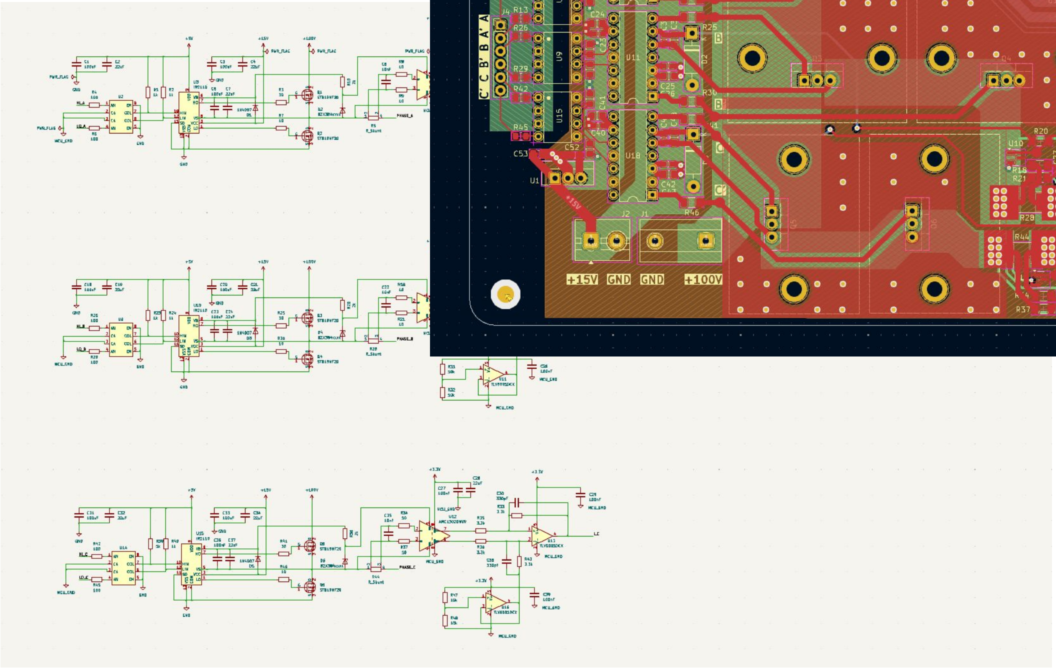
Demonstrate Precise Speed Control

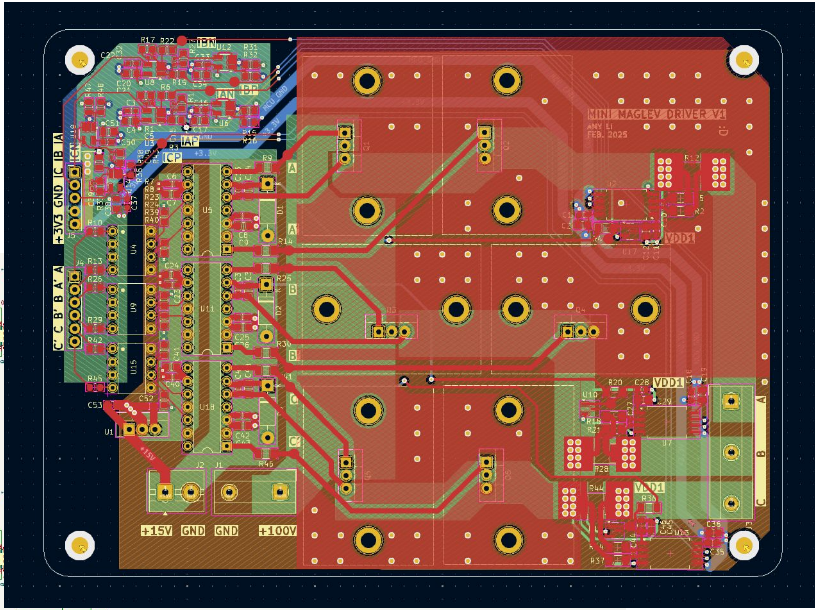


Progress…

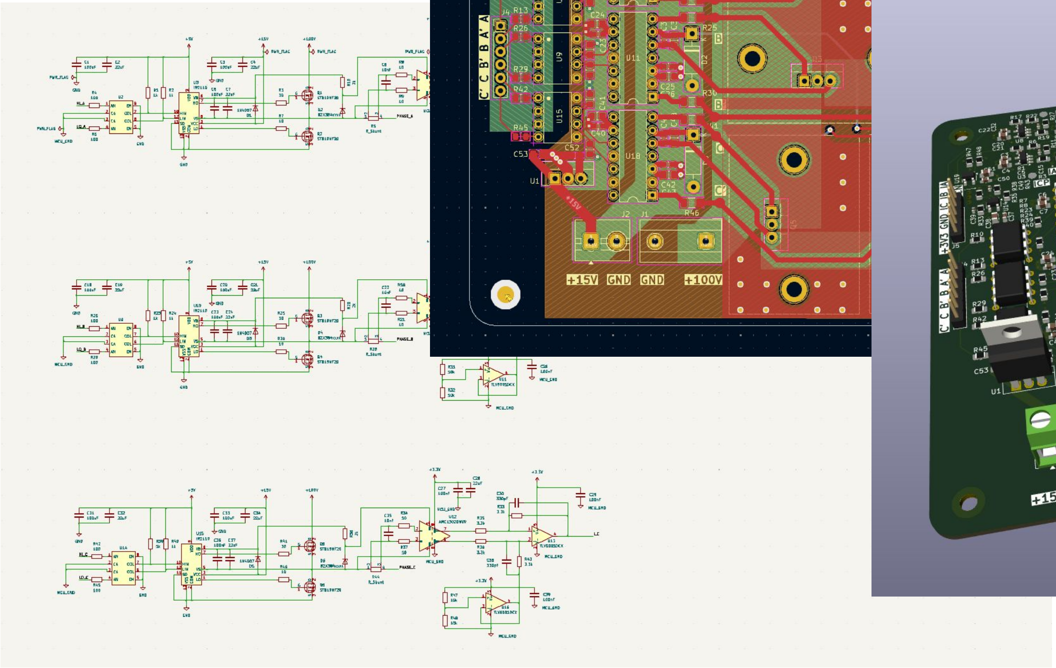


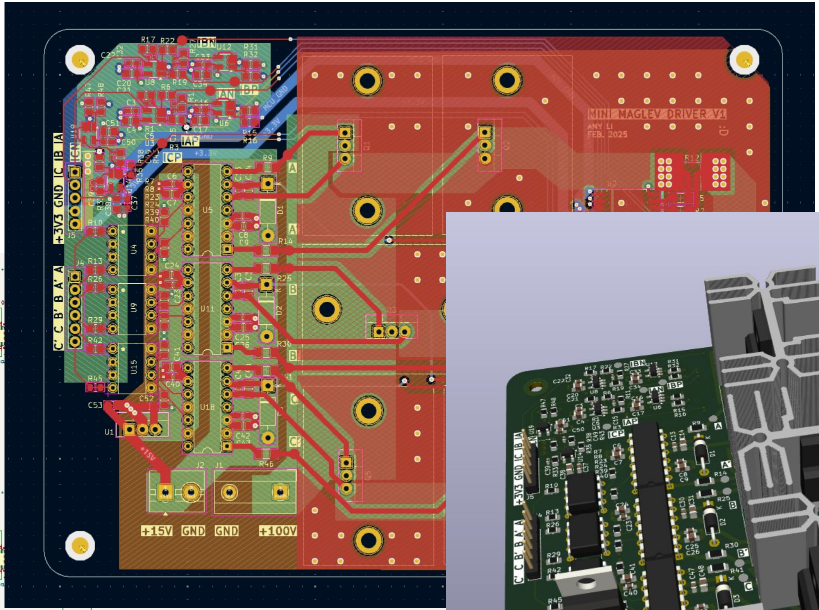
Progress…

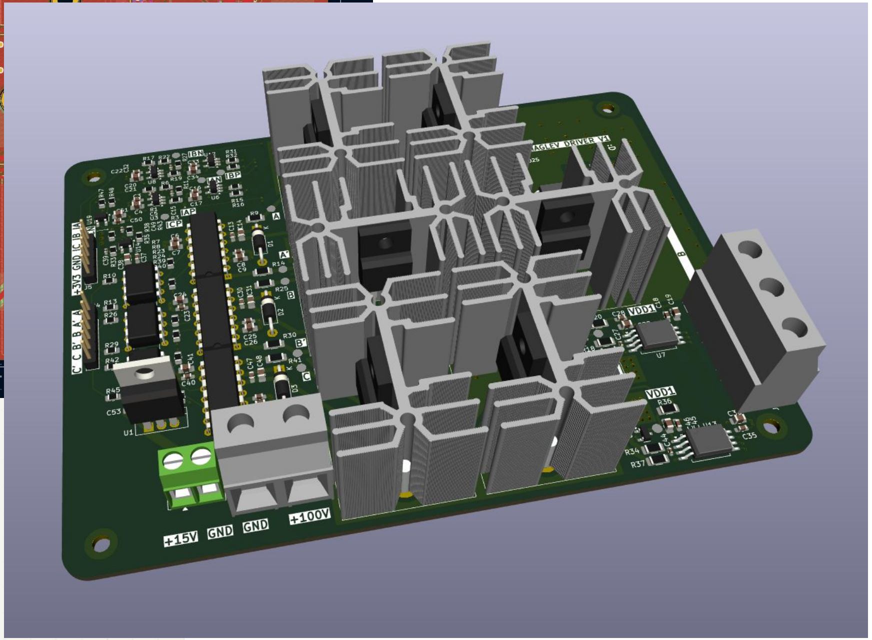




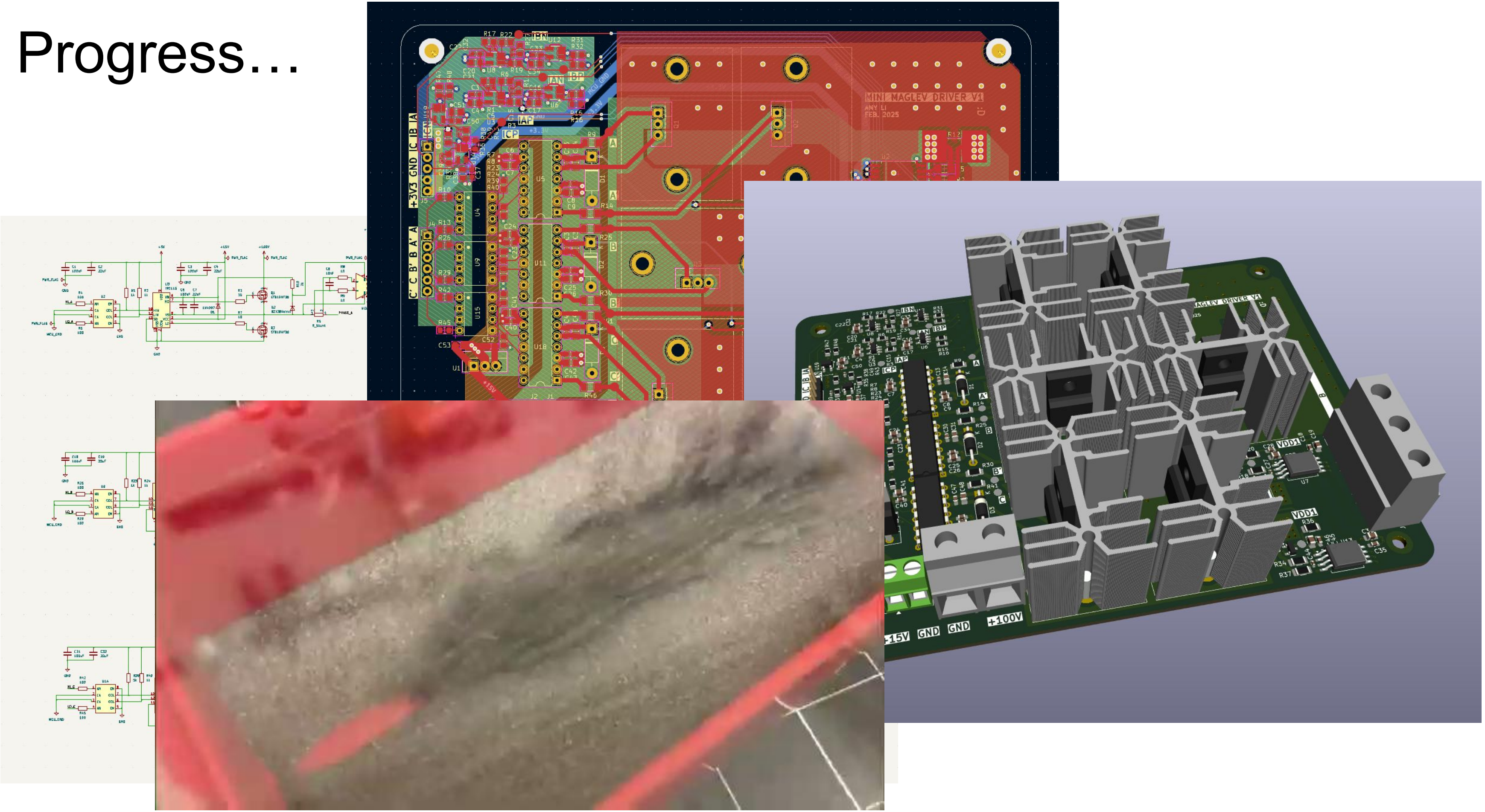
Progress…




MINI MAGLEV DRIVER V1
AMY LI
FEB. 2025
+15V
GND
GND
+100V


+15V
GND
GND
+100V

Progress…


MINI MAGLEV DRIVER V1
AMY LI
FEB. 2025
+3V3 GND IC IB IA
C' C B' B A' A
+15V GND GND +100V

Future work

Next steps for 2025

Linear induction motor propulsion

Full magnetic levitation

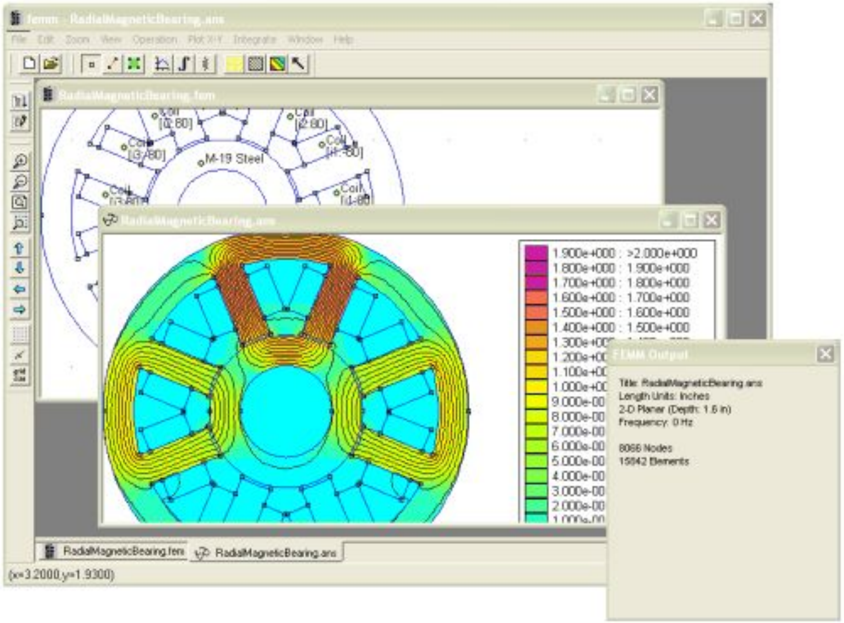
https://upload.wikimedia.org/wikipedia/commons/5/5e/Mark_III_SkyTrain_near_Nanaimo_station.jpg
https://en.wikipedia.org/wiki/Shanghai_maglev_train#/media/File:A_maglev_train_coming_out,_Pudong_International_Airport,_Shanghai.jpg

Future work: Stator design for maglev train

Experimenting with stator geometry for levitation

- E&M Simulation (FEMM)
- Simulink controls simulation

Finite Element Method Magnetics
https://www.femm.info/wiki/HomePage



Thank you for your attention!

Slides dump follows

Common-mode step responses? (**Ideal** step)

INA240 is less performant version of our INA241

Undamped ringing

Slow settling time (still underdamped)

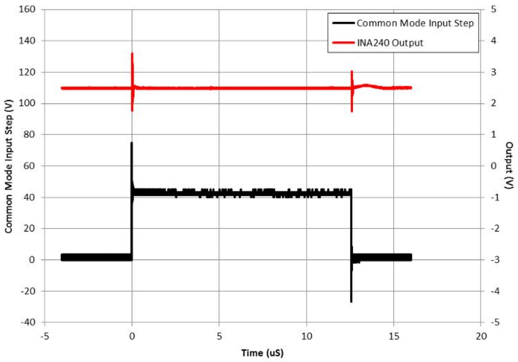
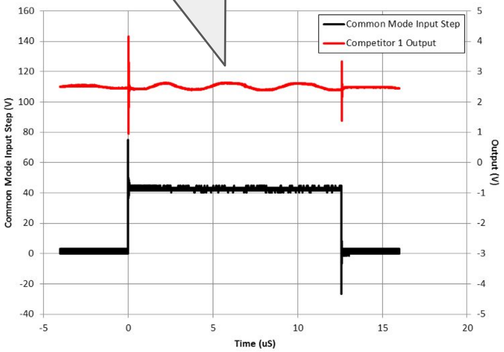


Figure 5. INA240 Output vs 40-V Common-Mode Step Input



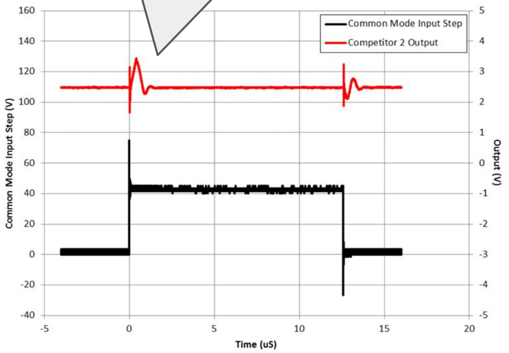
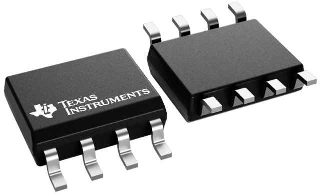


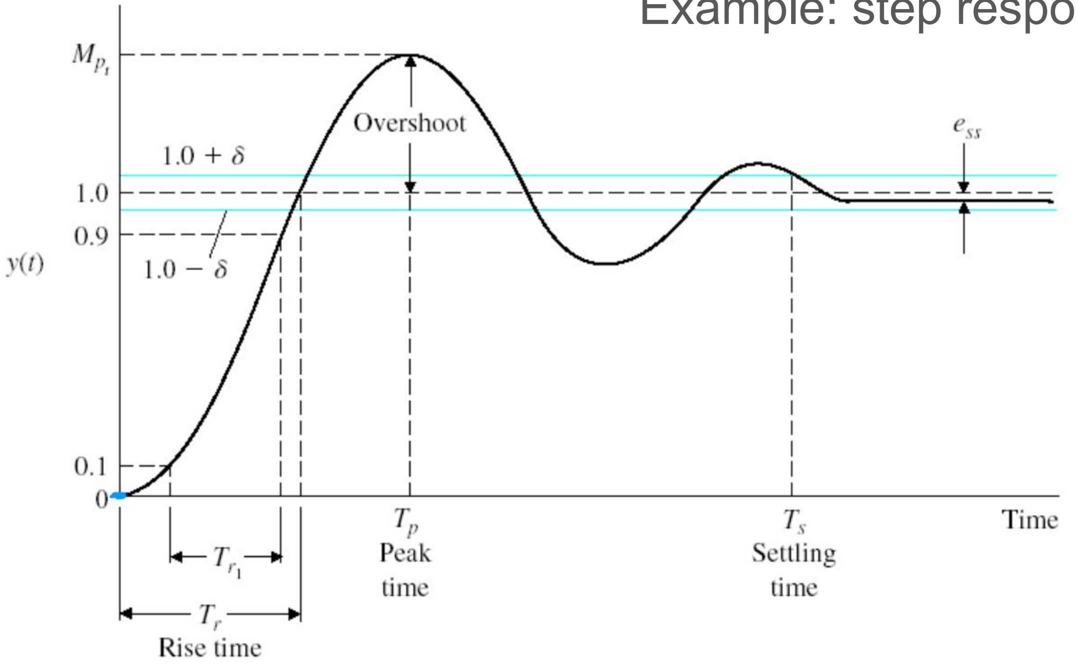
Figure 7. Competitor 2 Output vs 40-V Common-Mode Step Input

Source: [TI: Current sensing for inline motor-control applications]()

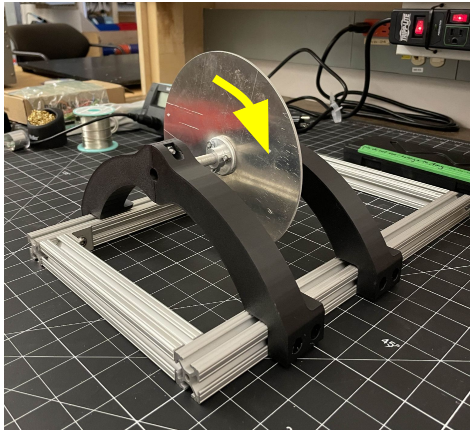


Term goal: Closed-loop **speed** control

Example: step response to speed setpoint



$\omega(t)$



Source: ELEC 341 *Notes_ResponseOf2ndOrderSystem.pdf*

Some sort of motivation (silly)

TGV ~ 320 km/h

[Train à Grande Vitesse | French railway system | Britannica](https://www.britannica.com/technology/maglev-train)

Source: https://www.britannica.com/technology/maglev-train

Some sort of motivation (silly)

TGV ~ 320 km/h

Image source

L0 Series ~ 600 km/h

Image source

Source: https://www.britannica.com/technology/maglev-train

Some sort of motivation (silly)

TGV ~ 320 km/h

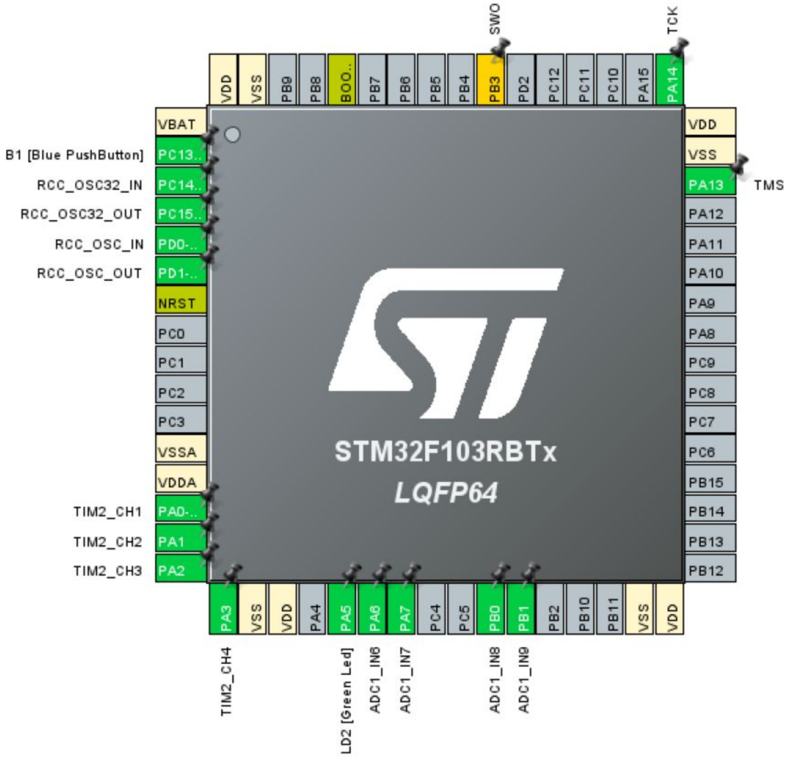
Image source

L0 Series ~ 600 km/h

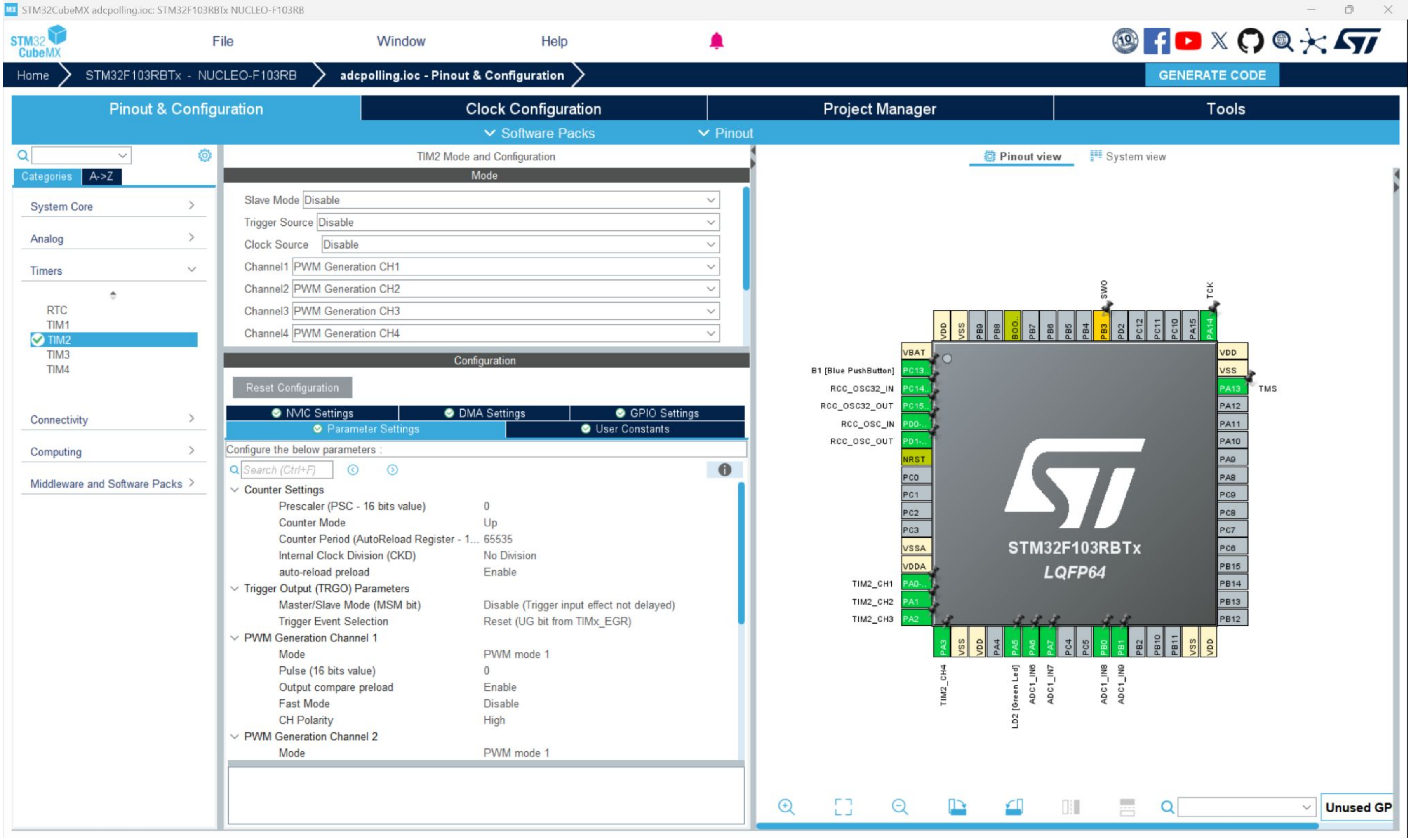
Image source

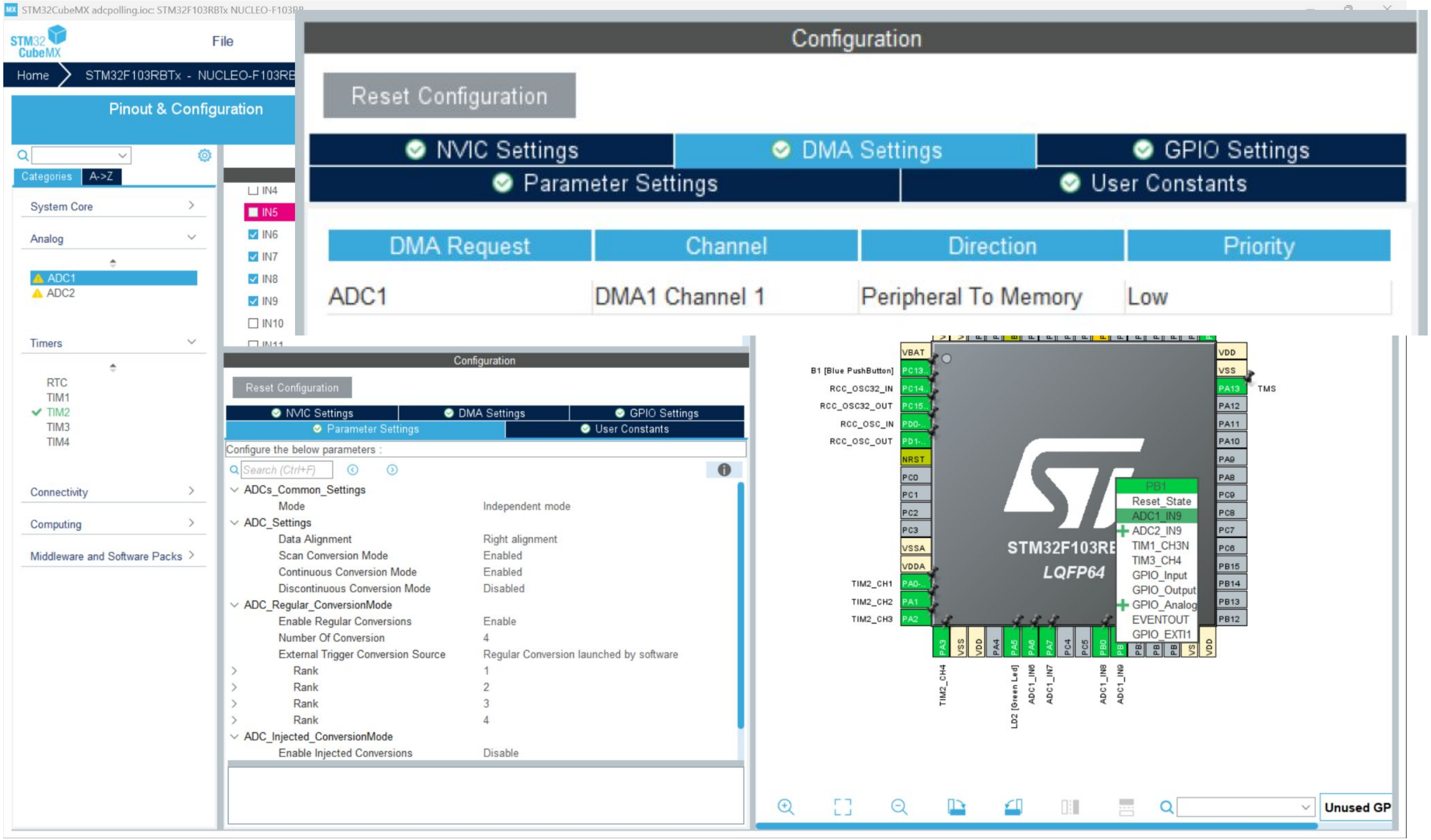
- Would be cool to have a small maglev running around Project Lab!

Source: https://www.britannica.com/technology/maglev-train



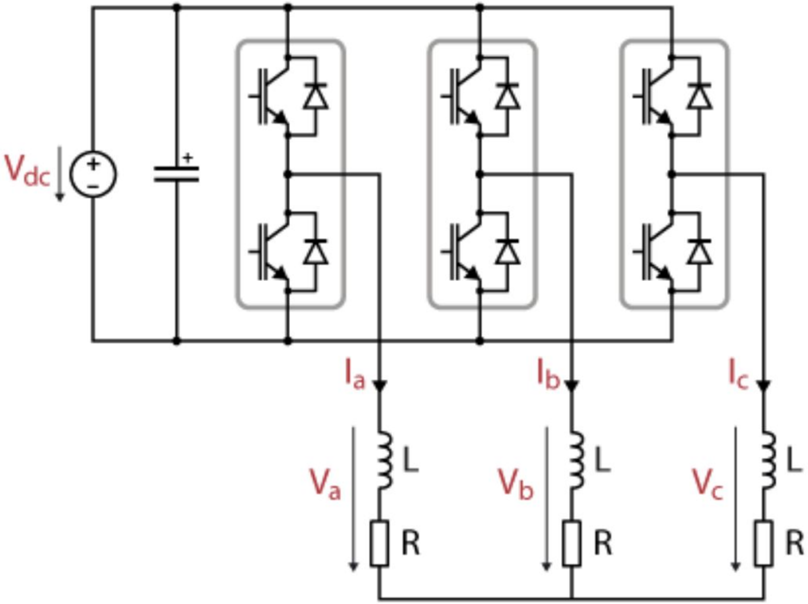

```c
#include "main.h"

ADC_HandleTypeDef hadc1;
DMA_HandleTypeDef hdma_adc1;
TIM_HandleTypeDef htim2;

uint32_t AD_RES_BUFFER[4];  // Circular buffer

void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_DMA_Init(void);
static void MX_ADC1_Init(void);
static void MX_TIM2_Init(void);

int main(void)
{
  HAL_Init();
  /* Configure the system clock */
  SystemClock_Config();
  /* Initialize all configured peripherals */
  MX_GPIO_Init();
  MX_DMA_Init();
  MX_ADC1_Init();
  MX_TIM2_Init();

  HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
  HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_2);
  HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_3);
  HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_4);
  HAL_ADC_Start_DMA(&hadc1, AD_RES_BUFFER, 4);

  while (1)
  {
      // Pipe ADC reading (12 bit) as CCR (16 bit) to TIM2
      TIM2->CCR1 = (AD_RES_BUFFER[0] << 4);
      TIM2->CCR2 = (AD_RES_BUFFER[1] << 4);
      TIM2->CCR3 = (AD_RES_BUFFER[2] << 4);
      TIM2->CCR4 = (AD_RES_BUFFER[3] << 4);
      HAL_Delay(1);
  }
}
```


STM32CubeMX adcpolling.ioc: STM32F103RBTx NUCLEO-F103RB
File
Window
Help
Home
STM32F103RBTx - NUCLEO-F103RB
adcpolling.ioc - Pinout & Configuration
GENERATE CODE
Pinout & Configuration
Clock Configuration
Project Manager
Tools
Software Packs
Pinout
Categories
A->Z
System Core
Analog
Timers
RTC
TIM1
TIM2
TIM3
TIM4
Connectivity
Computing
Middleware and Software Packs
TIM2 Mode and Configuration
Mode
Slave Mode Disable
Trigger Source Disable
Clock Source Disable
Channel1 PWM Generation CH1
Channel2 PWM Generation CH2
Channel3 PWM Generation CH3
Channel4 PWM Generation CH4
Configuration
Reset Configuration
NVIC Settings
DMA Settings
GPIO Settings
Parameter Settings
User Constants
Configure the below parameters :
Search (Ctrl+F)
Counter Settings
Prescaler (PSC - 16 bits value) 0
Counter Mode Up
Counter Period (AutoReload Register - 1... 65535
Internal Clock Division (CKD) No Division
auto-reload preload Enable
Trigger Output (TRGO) Parameters
Master/Slave Mode (MSM bit) Disable (Trigger input effect not delayed)
Trigger Event Selection Reset (UG bit from TIMx_EGR)
PWM Generation Channel 1
Mode PWM mode 1
Pulse (16 bits value) 0
Output compare preload Enable
Fast Mode Disable
CH Polarity High
PWM Generation Channel 2
Mode PWM mode 1
Pinout view
System view
STM32F103RBTx
LQFP64
SWO
TCK
TMS
B1 [Blue PushButton]
RCC_OSC32_IN
RCC_OSC32_OUT
RCC_OSC_IN
RCC_OSC_OUT
TIM2_CH1
TIM2_CH2
TIM2_CH3
TIM2_CH4
LD2 [Green Led]
ADC1_IN6
ADC1_IN7
ADC1_IN8
ADC1_IN9
Unused GP


STM32CubeMX adcpolling.ioc: STM32F103RBTx NUCLEO-F103RB
File
Home
STM32F103RBTx - NUCLEO-F103RB
Pinout & Configuration
Categories
A->Z
System Core
Analog
ADC1
ADC2
Timers
RTC
TIM1
TIM2
TIM3
TIM4
Connectivity
Computing
Middleware and Software Packs
IN4
IN5
IN6
IN7
IN8
IN9
IN10
Configuration
Reset Configuration
NVIC Settings
DMA Settings
GPIO Settings
Parameter Settings
User Constants
DMA Request
Channel
Direction
Priority
ADC1
DMA1 Channel 1
Peripheral To Memory
Low
Configure the below parameters :
ADCs_Common_Settings
Mode
Independent mode
ADC_Settings
Data Alignment
Right alignment
Scan Conversion Mode
Enabled
Continuous Conversion Mode
Enabled
Discontinuous Conversion Mode
Disabled
ADC_Regular_ConversionMode
Enable Regular Conversions
Enable
Number Of Conversion
4
External Trigger Conversion Source
Regular Conversion launched by software
Rank
1
Rank
2
Rank
3
Rank
4
ADC_Injected_ConversionMode
Enable Injected Conversions
Disable
B1 [Blue PushButton]
RCC_OSC32_IN
RCC_OSC32_OUT
RCC_OSC_IN
RCC_OSC_OUT
TIM2_CH1
TIM2_CH2
TIM2_CH3
TIM2_CH4
LD2 [Green Led]
ADC1_IN6
ADC1_IN7
ADC1_IN8
ADC1_IN9
TMS
STM32F103RB
LQFP64
PB1
Reset_State
ADC1_IN9
ADC2_IN9
TIM1_CH3N
TIM3_CH4
GPIO_Input
GPIO_Output
GPIO_Analog
EVENTOUT
GPIO_EXTI1
Unused GP

3-Phase Current Sensing Geometry

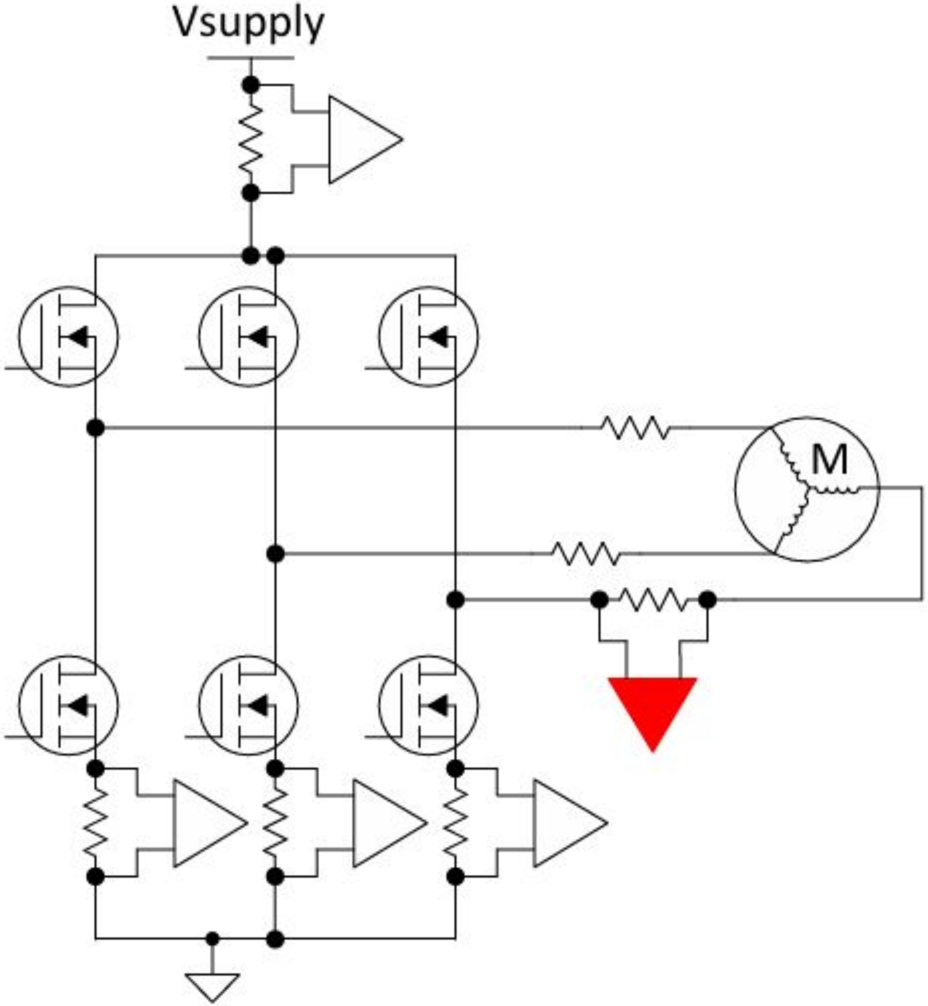
[TI Picture, maybe INA241 datasheet]



[TI: Current sensing for inline motor-control applications]

3-Phase (

[TI Picture]



TI: Current se

3-F

[TI I

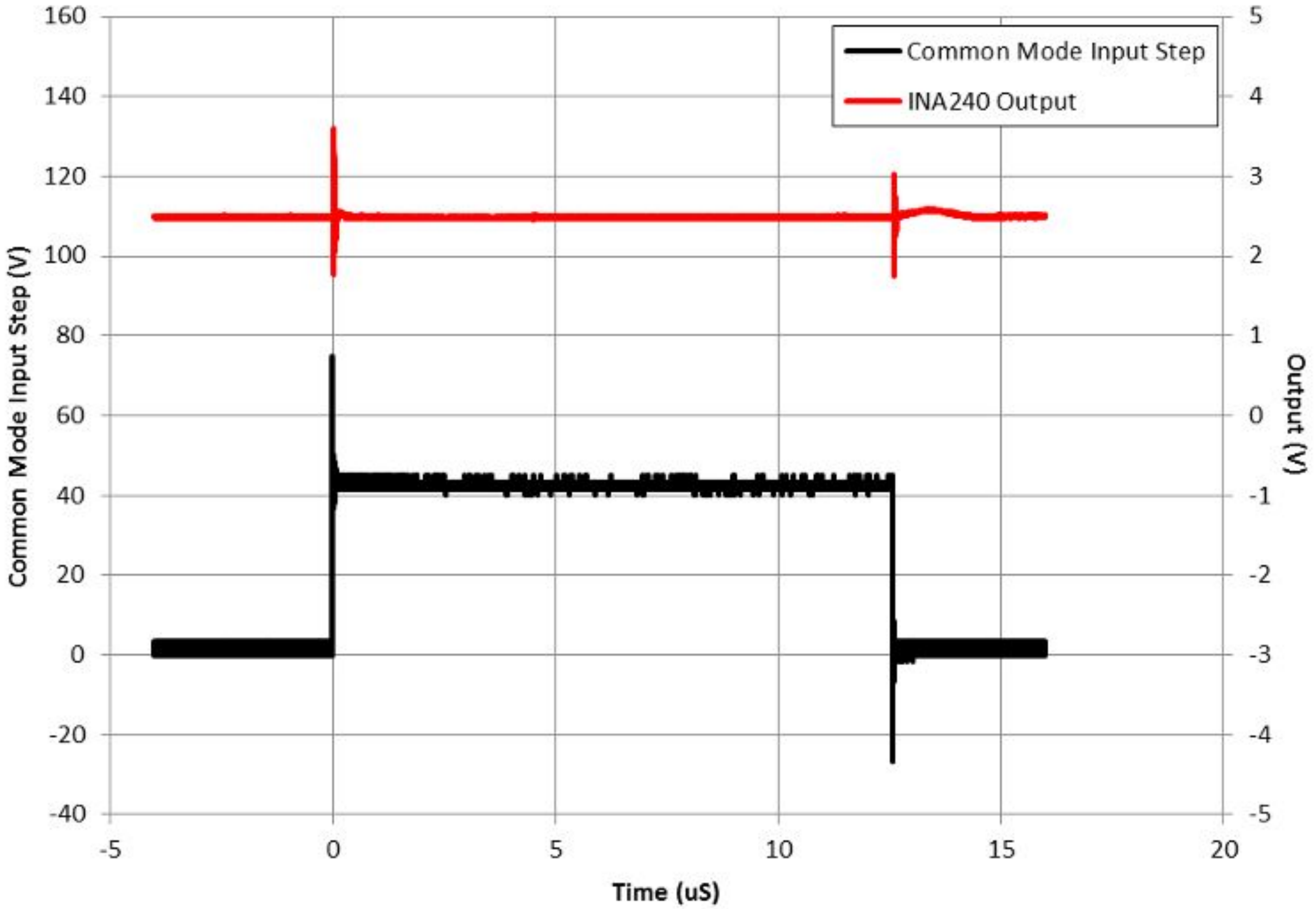


Figure 5. INA240 Output vs 40-V Common-Mode Step Input

TI: (

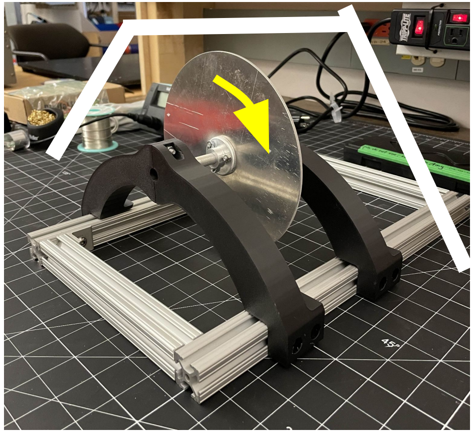
Instantaneous torque control

Introduce motivation: precise speed control rejecting disturbances

Introduce requirements:

1. Control of instantaneous current in 3 phases
2. Closed-loop current control with inline current sensing
3. 3-phase inverter, spatial vector modulation

Implement sensing & control system with STM32 Microcontroller



Mechanical energy (spinny!)

> **In what domains does energy exist in my system?**

Safety Systems

Ask Amy for image of thermocouple location on PCB schematic

High voltage protection: (image of power source & wooden board)

Example calculation: state update + current setpoint -> SVM voltage phasor

This needs to be done for our project soon anyway…